TÉCNICO
UNIVERSIDADE
DE LISBOA

Advanced Network Security and Architectures

METI

Report Module 1 [EN]

Authors:
José Oliveira (103771) jose.novais.oliveira@tecnico.ulisboa.pt
Afonso Mendes (116024) afonsofmendes@tecnico.ulisboa.pt
Tiago Videira (116069) tiago.g.videira@tecnico.ulisboa.pt

Group 3

2025/2026 – 2nd Semester, P4

Contents

1 Introduction

Computer networks are constantly exposed to a wide variety of security threats that may compromise confidentiality, integrity, and availability of services. Understanding how these attacks operate and how they can be mitigated is fundamental for the design of secure network infrastructures.

The objective of this project was to experimentally study several network security mechanisms and attacks using Cisco IOS devices, Linux hosts, GNS3, Wireshark, and different firewall and authentication technologies. The laboratory activities focused both on offensive techniques and defensive countermeasures, allowing a practical understanding of network security concepts.

The first part of the project addressed common Layer 2 and Layer 3 attacks, including MAC table overflow, DHCP spoofing, ARP poisoning, DNS spoofing, and RIP poisoning. For each attack, packet captures and device-level information were analyzed in order to understand the attack behavior and evaluate corresponding mitigation mechanisms such as port security, DHCP snooping, Dynamic ARP Inspection, HTTPS/TLS protections, and routing authentication.

The second part focused on firewall technologies and protection against Denial of Service attacks. Both Classical Firewalls (CBAC) and Zone-Based Policy Firewalls (ZBPF) were studied and configured using Cisco IOS. Stateful inspection, traffic classification, NAT, inter-zone security policies, and DoS mitigation mechanisms such as traffic policing and TCP session management were experimentally validated using Wireshark and NMAP.

Finally, the third part explored Authentication, Authorization, and Accounting (AAA) technologies. Centralized authentication using TACACS+ and RADIUS servers was implemented and analyzed, followed by IEEE 802.1X port-based network access control using EAP-PEAP authentication with FreeRADIUS and Cisco switches.

Throughout the project, Wireshark packet captures, Cisco IOS inspection commands, and practical experimentation were used to compare theoretical protocol behavior with real network observations. The performed experiments demonstrate both the feasibility of several common network attacks and the importance of properly configured security mechanisms in modern network infrastructures.

2 Network attacks and countermeasures

2.1 Objective

The main objective of this laboratory work is to study and experimentally evaluate a set of common network attacks and their corresponding countermeasures in a controlled environment using GNS3.

In particular, the following attacks are addressed:

- **MAC table overflow:** to understand how a switch can be forced to behave as a hub by saturating its MAC address table, and to evaluate mitigation mechanisms such as port security.
- **DHCP spoofing:** to demonstrate how a rogue DHCP server can provide malicious network configurations to clients, and to analyze the effectiveness of DHCP snooping as a countermeasure.
- **ARP poisoning (Man-in-the-Middle):** to perform a MitM attack by manipulating ARP tables, and to study the use of Dynamic ARP Inspection (DAI) for protection.
- **DNS spoofing:** to analyze how a victim can be redirected to a fake web server through DNS manipulation combined with ARP poisoning.
- **RIP poisoning:** to investigate how routing tables can be corrupted using forged RIP messages, leading to traffic redirection, and to explore authentication-based defenses.

Additionally, this work aims to:

- Analyze network behavior through packet captures using Wireshark;
- Interpret device-level information using Cisco IOS commands;
- Evaluate the practical feasibility and limitations of each attack;
- Compare experimental results with theoretical expectations.

2.2 MAC Table Overflow

2.2.1 Attack and Countermeasure

The MAC table overflow attack was performed using the `camov.py` script, generating a large number of frames with different spoofed MAC addresses.

As observed in Wireshark (Figure 1), a high volume of traffic with varying source addresses was captured, indicating the flooding behavior of the attack.

The switch behavior was analyzed using `show mac address-table`.

To mitigate the attack, port security was configured, limiting the number of MAC addresses per port and preventing the insertion of multiple spoofed entries.

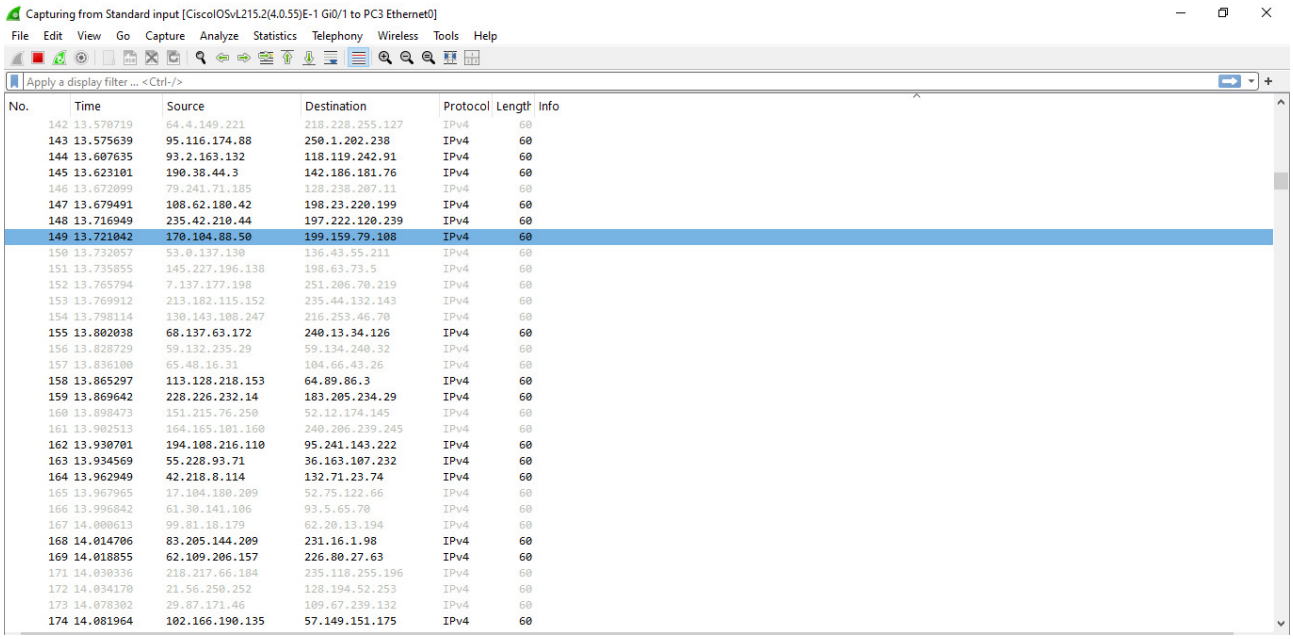


Figure 1: Wireshark capture during MAC table overflow attack

2.2.2 Switch Comparison

Cisco Catalyst 2960 supports port security, allowing the configuration of a maximum number of MAC addresses per port and defining actions when violations occur (e.g., restrict or shutdown). This makes it possible to prevent MAC table overflow attacks.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

In contrast, the TP-Link TL-SF1005D is an unmanaged switch that does not support port security or any traffic control mechanisms, as it lacks a configuration interface. Therefore, it cannot prevent MAC flooding attacks.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.2.3 Feasibility

The objective of this attack is to flood the switch MAC address table, forcing the switch to behave like a hub. In that state, frames may be forwarded to all ports, allowing the attacker to listen to conversations between other hosts.

However, in practice, the attack may be difficult to execute successfully on modern switches because their MAC address tables are large. Additionally, the high amount of generated traffic can cause network instability or a denial-of-service condition, instead of allowing the attacker to reliably capture useful traffic.

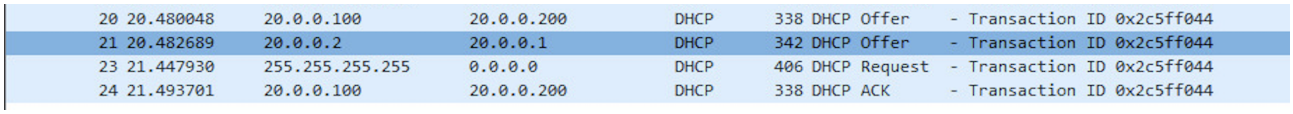
Therefore, although the attack is theoretically possible, its practical feasibility is limited and depends on the switch capacity, configuration, and security features.

2.3 DHCP spoofing

2.3.1 Attack and Countermeasure

The DHCP spoofing attack was performed by introducing a rogue DHCP server that responds to DHCP Discover messages with malicious configuration parameters.

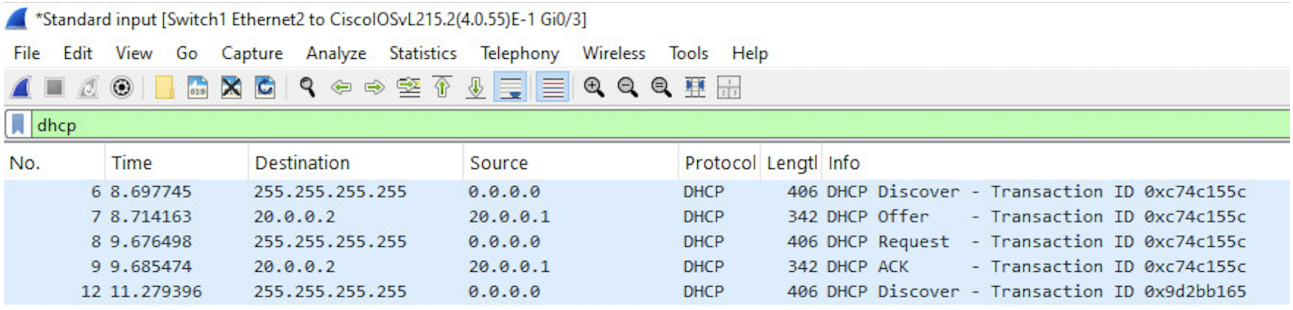
As observed in Wireshark (Figure 2), multiple DHCP Offer messages were received for the same transaction ID, indicating a race condition between the legitimate and the rogue DHCP servers. The victim accepts the first Offer received, which may result in a malicious gateway configuration provided by the attacker.



20	20.480048	20.0.0.100	20.0.0.200	DHCP	338	DHCP Offer - Transaction ID 0x2c5ff044
21	20.482689	20.0.0.2	20.0.0.1	DHCP	342	DHCP Offer - Transaction ID 0x2c5ff044
23	21.447930	255.255.255.255	0.0.0.0	DHCP	406	DHCP Request - Transaction ID 0x2c5ff044
24	21.493701	20.0.0.100	20.0.0.200	DHCP	338	DHCP ACK - Transaction ID 0x2c5ff044

Figure 2: Wireshark capture showing multiple DHCP Offer messages (attack)

After enabling DHCP snooping and configuring the port connected to the legitimate DHCP server as trusted, only one DHCP Offer was observed (Figure 3). The rogue DHCP responses were blocked by the switch, and the client received a valid configuration. This confirms that DHCP snooping effectively mitigates the attack.



No.	Time	Destination	Source	Protocol	Lengtl	Info
6	8.697745	255.255.255.255	0.0.0.0	DHCP	406	DHCP Discover - Transaction ID 0xc74c155c
7	8.714163	20.0.0.2	20.0.0.1	DHCP	342	DHCP Offer - Transaction ID 0xc74c155c
8	9.676498	255.255.255.255	0.0.0.0	DHCP	406	DHCP Request - Transaction ID 0xc74c155c
9	9.685474	20.0.0.2	20.0.0.1	DHCP	342	DHCP ACK - Transaction ID 0xc74c155c
12	11.279396	255.255.255.255	0.0.0.0	DHCP	406	DHCP Discover - Transaction ID 0x9d2bb165

Figure 3: Wireshark capture with DHCP snooping enabled (only legitimate Offer)

2.3.2 AI Prediction

The AI predicted that the victim closer to the attacker would be more likely to be compromised, since the rogue DHCP Offer would arrive first. This prediction matched the experimental results, where the victim receiving the first Offer obtained the malicious configuration.

2.3.3 Switch Comparison

Cisco Catalyst 2960 supports DHCP snooping, allowing ports to be classified as trusted or untrusted and blocking DHCP server messages from untrusted ports.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

The TP-Link TL-SF1005D does not support DHCP snooping, as it is an unmanaged switch without configuration capabilities or security features.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.3.4 Feasibility

The attack is feasible in networks without DHCP snooping, especially when the attacker is closer to the victim than the legitimate DHCP server. However, due to the race condition, success is not guaranteed. When DHCP snooping is enabled, the attack becomes ineffective.

2.4 ARP poisoning (MitM)

2.4.1 Network Topology

The ARP poisoning attack was performed in the network shown in Figure 4, where two hosts communicate through a switch and an attacker is connected to the same Layer 2 domain.

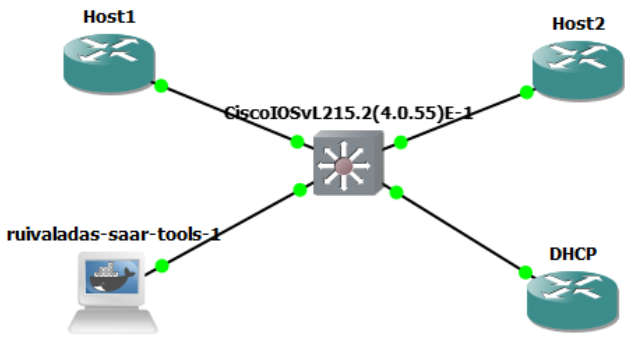


Figure 4: Network topology for ARP poisoning attack

2.4.2 Attack and Countermeasure

As observed in Wireshark (Figure 5), ARP packets indicate duplicate IP usage, confirming that the attacker is associating its MAC address with multiple IP addresses.

At the same time, ICMP traffic between the victims is still observed, which demonstrates that communication is maintained while being intercepted by the attacker, confirming a successful Man-in-the-Middle (MitM) condition.

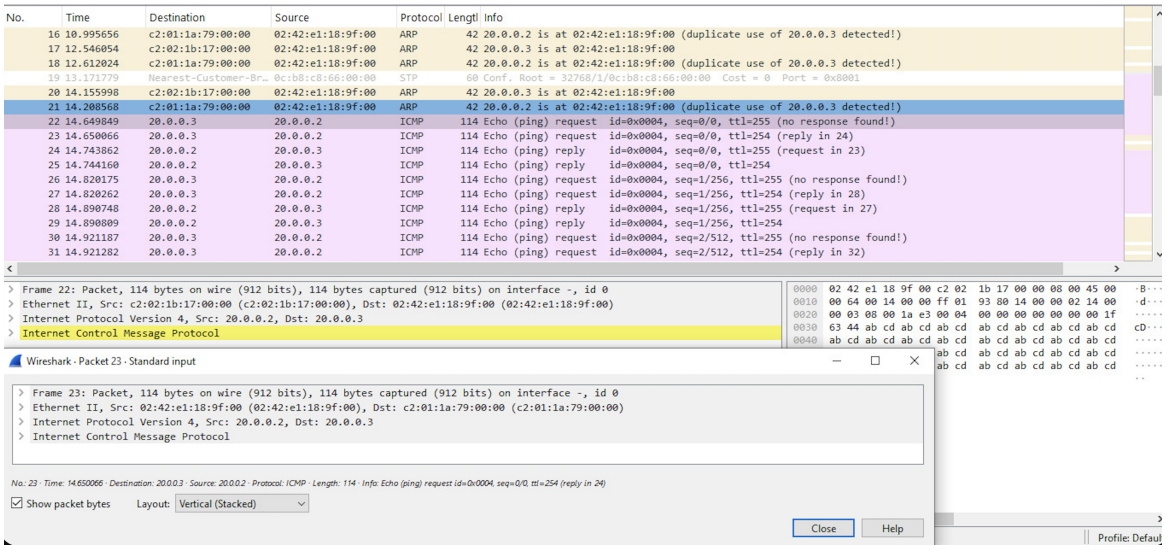


Figure 5: ARP poisoning showing duplicate IP detection and intercepted ICMP traffic

The attacker generated multiple forged ARP requests and replies, attempting to poison the ARP tables of the victims, as shown in Figure 6.

22	41.442250	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.3? Tell 20.0.0.2
24	43.473802	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.4? Tell 20.0.0.2
26	45.523090	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.3? Tell 20.0.0.2
28	47.548118	Broadcast	02:42:e1:18:9f:00	ARP	42 20.0.0.4 is at 02:42:e1:18:9f:00
29	47.597578	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.4? Tell 20.0.0.2
31	49.613918	Broadcast	02:42:e1:18:9f:00	ARP	42 20.0.0.3 is at 02:42:e1:18:9f:00
32	51.161927	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.3? Tell 20.0.0.2
34	53.178241	Broadcast	02:42:e1:18:9f:00	ARP	42 20.0.0.4 is at 02:42:e1:18:9f:00
35	53.226972	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.4? Tell 20.0.0.2
37	55.245424	Broadcast	02:42:e1:18:9f:00	ARP	42 20.0.0.3 is at 02:42:e1:18:9f:00
41	56.794383	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.3? Tell 20.0.0.2
43	58.809905	Broadcast	02:42:e1:18:9f:00	ARP	42 20.0.0.4 is at 02:42:e1:18:9f:00
44	58.862239	Broadcast	02:42:e1:18:9f:00	ARP	42 Who has 20.0.0.4? Tell 20.0.0.2

Figure 6: ARP packets generated by the attacker

However, the attack was mitigated using Dynamic ARP Inspection (DAI). The switch detected invalid ARP packets and blocked them, as shown in the system logs (Figure 7).

```
*Apr 29 10:34:18.724: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.3/10:34:18 UTC Wed Apr 29 2026])
*Apr 29 10:34:20.876: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.4/0000.000
0.0000/20.0.0.3/10:34:20 UTC Wed Apr 29 2026])
*Apr 29 10:34:20.877: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.4/10:34:20 UTC Wed Apr 29 2026])
*Apr 29 10:34:23.095: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.3/0000.000
0.0000/20.0.0.4/10:34:21 UTC Wed Apr 29 2026])
*Apr 29 10:34:24.097: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.3/10:34:23 UTC Wed Apr 29 2026])
*Apr 29 10:34:25.279: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Res) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.4/0000.000
0.0000/20.0.0.3/10:34:25 UTC Wed Apr 29 2026])
*Apr 29 10:34:25.280: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi0/0, vlan 1.([0242.e118.9f00/20.0.0.2/0000.000
0.0000/20.0.0.4/10:34:25 UTC Wed Apr 29 2026])
*Apr 29 10:34:26.622: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on GigabitEthernet0/3 (not half duplex), with DHCP
FastEthernet0/0 (half duplex).
```

Figure 7: DAI log showing invalid ARP packets being denied

2.4.3 Defense Statistics

The effectiveness of DAI is confirmed by the statistics shown in Figure 8, where a high number of ARP packets were dropped by the switch.

```
Switch#show ip arp inspection statistics

Vlan      Forwarded        Dropped     DHCP Drops      ACL Drops
----      ---------        -------     ----------      ---------
   1              6            259            259              0

Vlan   DHCP Permits    ACL Permits  Probe Permits   Source MAC Failures
----   ------------    -----------  -------------   -------------------
   1              6              0              0                     0

Vlan   Dest MAC Failures   IP Validation Failures   Invalid Protocol Data
----   -----------------   ----------------------   ---------------------
   1                   0                        0                       0
```

Figure 8: DAI statistics showing dropped ARP packets

2.4.4 AI Configuration vs Experimental Setup

The AI suggested enabling DHCP snooping and configuring Dynamic ARP Inspection on the switch, marking trusted interfaces accordingly.

This matched the experimental setup, where DHCP snooping was required for DAI to validate ARP packets. The configuration had to be adapted to the topology, particularly in defining trusted ports and VLAN configuration.

2.4.5 Switch Comparison

Cisco Catalyst 2960 supports Dynamic ARP Inspection, allowing validation of ARP packets based on DHCP snooping bindings.

Link: www.cisco.com/c/en/us/products/switches/catalyst-2960-series-switches/index.html

The TP-Link TL-SF1005D does not support DAI, as it is an unmanaged switch without security or inspection mechanisms.

Link: www.tp-link.com/en/home-networking/soho-switch/tl-sf1005d/

2.4.6 Feasibility

The attacker attempted to poison the ARP tables by sending forged ARP messages. The attack successfully created a Man-in-the-Middle condition, as observed in Wireshark. However, based on the switch logs and DAI statistics, these packets were detected and blocked.

Therefore, although the attack is feasible in networks without protection, in this setup it was not successful due to the correct configuration of Dynamic ARP Inspection.

2.5 DNS spoofing

2.5.1 Attack Description

The attack combined ARP poisoning with DNS spoofing to redirect the victim's traffic to a malicious host.

Initially, the attacker poisoned the ARP caches of the victim and the gateway by repeatedly sending forged ARP replies claiming ownership of multiple IP addresses.

As a result, the victim associated the attacker's MAC address with the IP addresses of legitimate hosts on the network, allowing the attacker to position itself as a man-in-the-middle.

After intercepting the traffic, the attacker responded to DNS queries with forged DNS responses containing malicious IP addresses.

The victim then attempted to establish TCP/TLS connections towards the attacker-controlled destination instead of the legitimate server.

2.5.2 Analysis of ARP, DNS and TCP Packets

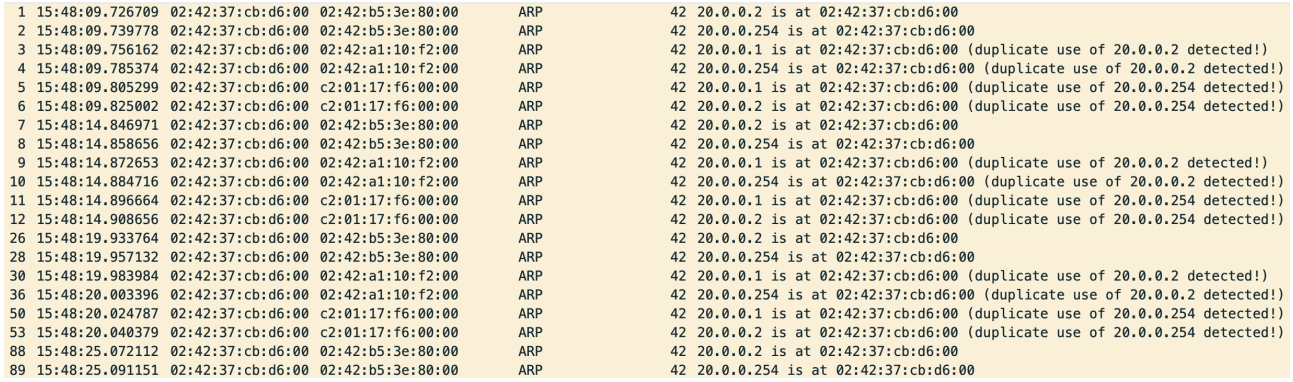
The attack started with ARP poisoning packets used to manipulate the victim's ARP cache.

Once positioned as a man-in-the-middle, the attacker intercepted DNS requests and generated forged DNS replies redirecting the victim to a malicious IP address.

The victim then attempted to establish TCP/TLS connections towards the spoofed destination.

The Wireshark captures show the full sequence of events:

1. Forged ARP replies poison the ARP cache
2. DNS requests are intercepted
3. Spoofed DNS replies redirect the victim
4. TCP/TLS connections are attempted
5. TLS communication fails due to certificate mismatch



1	15:48:09.726709	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00
2	15:48:09.739778	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00
3	15:48:09.756162	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
4	15:48:09.785374	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
5	15:48:09.805299	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
6	15:48:09.825002	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
7	15:48:14.846971	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00
8	15:48:14.858656	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00
9	15:48:14.872653	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
10	15:48:14.884716	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
11	15:48:14.896664	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
12	15:48:14.908656	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
26	15:48:19.933764	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00
28	15:48:19.957132	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00
30	15:48:19.983984	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
36	15:48:20.003396	02:42:37:cb:d6:00	02:42:a1:10:f2:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.2 detected!)
50	15:48:20.024787	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.1 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
53	15:48:20.040379	02:42:37:cb:d6:00	c2:01:17:f6:00:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00 (duplicate use of 20.0.0.254 detected!)
88	15:48:25.072112	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.2 is at 02:42:37:cb:d6:00
89	15:48:25.091151	02:42:37:cb:d6:00	02:42:b5:3e:80:00	ARP	42	20.0.0.254 is at 02:42:37:cb:d6:00

Figure 9: Wireshark capture showing repeated forged ARP replies

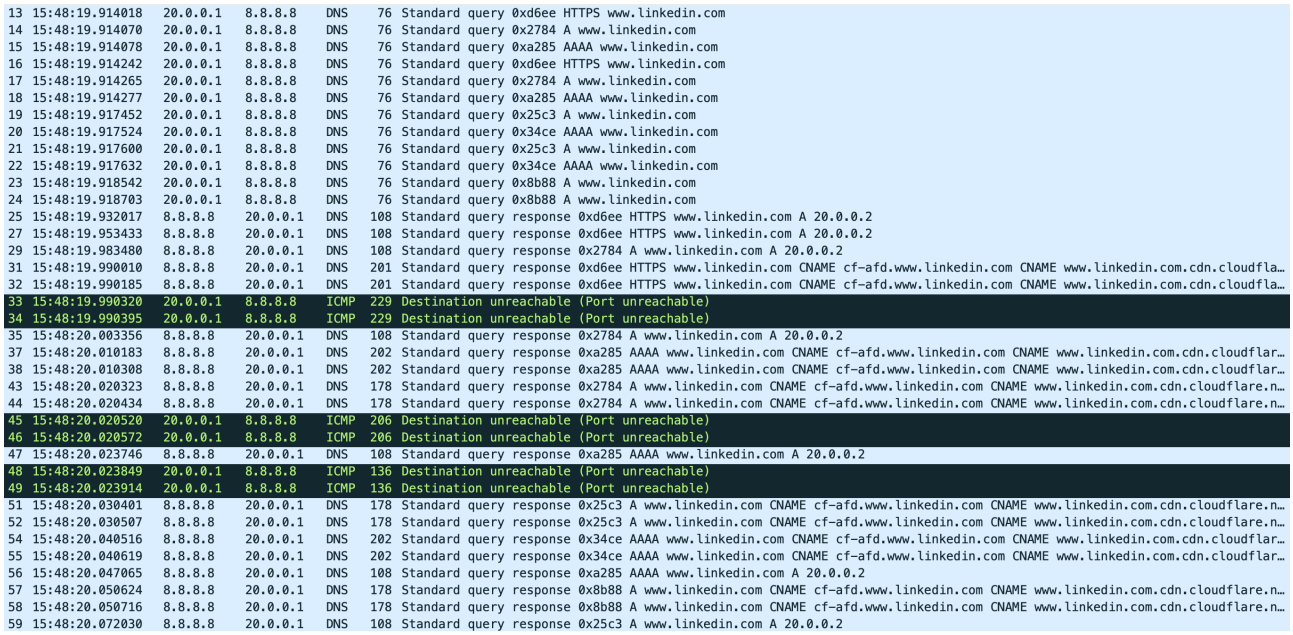
The ARP capture shows multiple unsolicited ARP replies claiming that several IP addresses correspond to the same MAC address.

Messages such as:

```
20.0.0.2 is at 02:42:37:cb:d6:00
20.0.0.254 is at 02:42:37:cb:d6:00
```

indicate that the attacker continuously poisoned the ARP caches of the victim and gateway.

Wireshark also detected duplicate IP usage warnings, which is a strong indication of ARP spoofing activity.



13	15:48:19.914018	20.0.0.1	8.8.8.8	DNS	76	Standard query 0xd6ee HTTPS www.linkedin.com
14	15:48:19.914070	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x2784 A www.linkedin.com
15	15:48:19.914078	20.0.0.1	8.8.8.8	DNS	76	Standard query 0xa285 AAAA www.linkedin.com
16	15:48:19.914242	20.0.0.1	8.8.8.8	DNS	76	Standard query 0xd6ee HTTPS www.linkedin.com
17	15:48:19.914265	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x2784 A www.linkedin.com
18	15:48:19.914277	20.0.0.1	8.8.8.8	DNS	76	Standard query 0xa285 AAAA www.linkedin.com
19	15:48:19.917452	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x25c3 A www.linkedin.com
20	15:48:19.917524	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x34ce AAAA www.linkedin.com
21	15:48:19.917600	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x25c3 A www.linkedin.com
22	15:48:19.917632	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x34ce AAAA www.linkedin.com
23	15:48:19.918542	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x8b88 A www.linkedin.com
24	15:48:19.918703	20.0.0.1	8.8.8.8	DNS	76	Standard query 0x8b88 A www.linkedin.com
25	15:48:19.932017	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0xd6ee HTTPS www.linkedin.com A 20.0.0.2
27	15:48:19.953433	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0xd6ee HTTPS www.linkedin.com A 20.0.0.2
29	15:48:19.983480	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0x2784 A www.linkedin.com A 20.0.0.2
31	15:48:19.990010	8.8.8.8	20.0.0.1	DNS	201	Standard query response 0xd6ee HTTPS www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudfla…
32	15:48:19.990185	8.8.8.8	20.0.0.1	DNS	201	Standard query response 0xd6ee HTTPS www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudfla…
33	15:48:19.990320	20.0.0.1	8.8.8.8	ICMP	229	Destination unreachable (Port unreachable)
34	15:48:19.990395	20.0.0.1	8.8.8.8	ICMP	229	Destination unreachable (Port unreachable)
35	15:48:20.003356	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0x2784 A www.linkedin.com A 20.0.0.2
37	15:48:20.010183	8.8.8.8	20.0.0.1	DNS	202	Standard query response 0xa285 AAAA www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflar…
38	15:48:20.010308	8.8.8.8	20.0.0.1	DNS	202	Standard query response 0xa285 AAAA www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflar…
43	15:48:20.020323	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x2784 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
44	15:48:20.020434	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x2784 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
45	15:48:20.020520	20.0.0.1	8.8.8.8	ICMP	206	Destination unreachable (Port unreachable)
46	15:48:20.020572	20.0.0.1	8.8.8.8	ICMP	206	Destination unreachable (Port unreachable)
47	15:48:20.023746	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0xa285 AAAA www.linkedin.com A 20.0.0.2
48	15:48:20.023849	20.0.0.1	8.8.8.8	ICMP	136	Destination unreachable (Port unreachable)
49	15:48:20.023914	20.0.0.1	8.8.8.8	ICMP	136	Destination unreachable (Port unreachable)
51	15:48:20.030401	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x25c3 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
52	15:48:20.030507	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x25c3 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
54	15:48:20.040516	8.8.8.8	20.0.0.1	DNS	202	Standard query response 0x34ce AAAA www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflar…
55	15:48:20.040619	8.8.8.8	20.0.0.1	DNS	202	Standard query response 0x34ce AAAA www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflar…
56	15:48:20.047065	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0xa285 AAAA www.linkedin.com A 20.0.0.2
57	15:48:20.050624	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x8b88 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
58	15:48:20.050716	8.8.8.8	20.0.0.1	DNS	178	Standard query response 0x8b88 A www.linkedin.com CNAME cf-afd.www.linkedin.com CNAME www.linkedin.com.cdn.cloudflare.n…
59	15:48:20.072030	8.8.8.8	20.0.0.1	DNS	108	Standard query response 0x25c3 A www.linkedin.com A 20.0.0.2

Figure 10: Forged DNS responses observed in Wireshark

The DNS capture shows queries for the domain `www.linkedin.com` followed by forged DNS responses.

Instead of returning the legitimate public IP address, the spoofed DNS responses mapped the domain name to the malicious local IP address:

```
www.linkedin.com A 20.0.0.2
```

This demonstrates that the attacker successfully intercepted and manipulated DNS traffic.

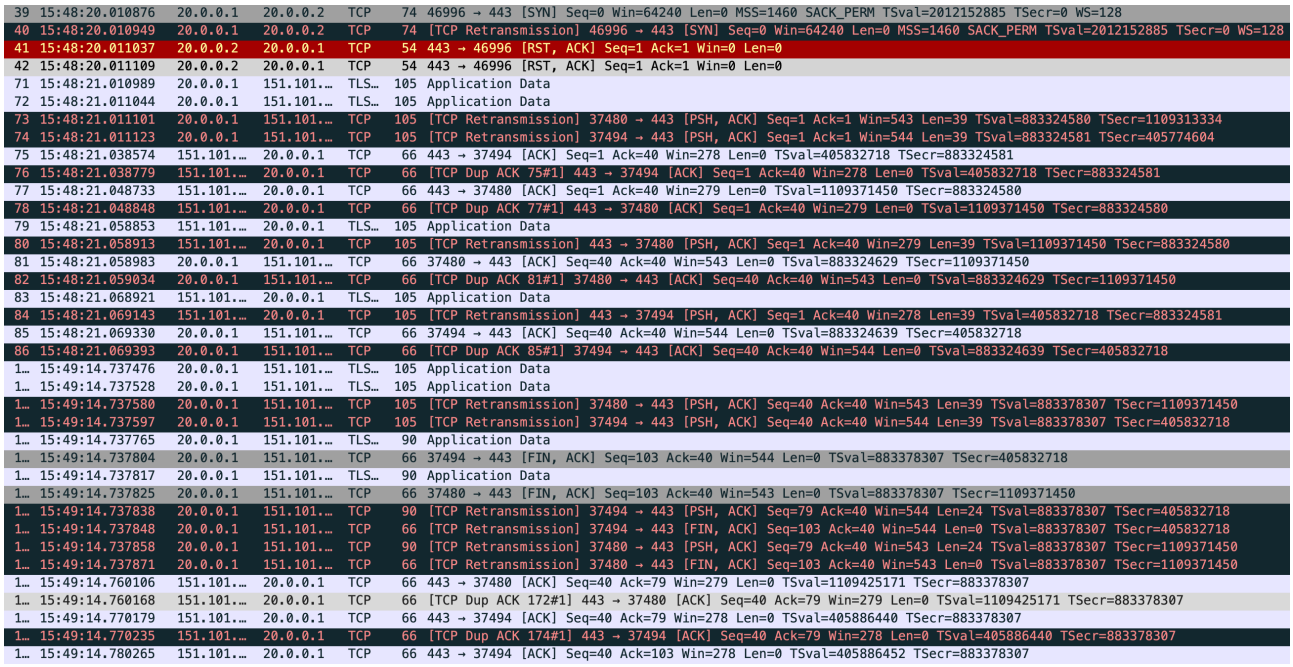


Figure 11: TCP and TLS communication after DNS spoofing

After the spoofed DNS resolution, the victim attempted to establish TCP and TLS sessions with the attacker-controlled host.

However, the packet capture shows multiple TCP retransmissions, duplicate acknowledgements, and TCP reset packets.

This behavior indicates that the TLS session could not be properly established.

The failure likely occurred because the attacker did not possess a valid TLS certificate for the spoofed domain, causing the browser or TLS stack to reject the connection.

2.5.3 AI Explanation and Experimental Comparison

An AI-generated explanation suggested that DNS spoofing may fail even when ARP poisoning is successful because modern browsers and operating systems implement additional security mechanisms such as:

- DNS caching
- HTTPS and TLS certificate validation
- HSTS policies

- DNS-over-HTTPS (DoH)
- Browser security protections

The experimental observations largely matched this explanation.

Although ARP poisoning was clearly successful, as demonstrated by the repeated forged ARP replies, the final TCP/TLS communication did not fully succeed.

The Wireshark captures show TCP retransmissions, duplicate ACKs, and TCP reset packets after the spoofed DNS replies.

This suggests that the victim attempted to establish a secure HTTPS connection but rejected the spoofed server because it could not provide a valid TLS certificate for the requested domain.

Therefore, the experiment confirms that successful ARP poisoning alone does not guarantee successful DNS spoofing attacks against modern HTTPS-protected services.

The roles of the entities involved in the authentication process were also correctly explained by the AI tool:

- The victim host acted as the DNS client
- The attacker acted as a man-in-the-middle through ARP poisoning
- The DNS server was impersonated through forged responses

These behaviors were directly observable in the packet captures.

2.5.4 Feasibility of the Attack

The experiment demonstrates that ARP poisoning and DNS spoofing attacks are technically feasible in local networks where hosts trust ARP replies without authentication.

However, the practical effectiveness of the attack against modern websites is significantly reduced by HTTPS and TLS protections.

Even though DNS manipulation succeeded, the browser and TLS mechanisms prevented the attacker from fully impersonating the legitimate website.

The attack would be significantly more effective against:

- Unencrypted HTTP services
- Legacy devices
- Misconfigured networks
- Users ignoring TLS certificate warnings

2.5.5 Countermeasures

Several mechanisms can mitigate ARP poisoning and DNS spoofing attacks:

- Static ARP entries
- Dynamic ARP Inspection (DAI)
- DHCP Snooping
- DNSSEC
- HTTPS and TLS certificate validation
- HSTS
- VPN usage
- Network segmentation
- Intrusion detection systems

Modern HTTPS deployments significantly reduce the effectiveness of DNS spoofing attacks, since attackers cannot easily present valid TLS certificates for legitimate domains.

2.6 RIP Poisoning

2.6.1 Network Topology

The RIP poisoning attack was performed using the topology shown in Figure 12. The Web browser, the legitimate Web server and the fake Web server are located in different subnets, with the attacker collocated with the fake Web server.

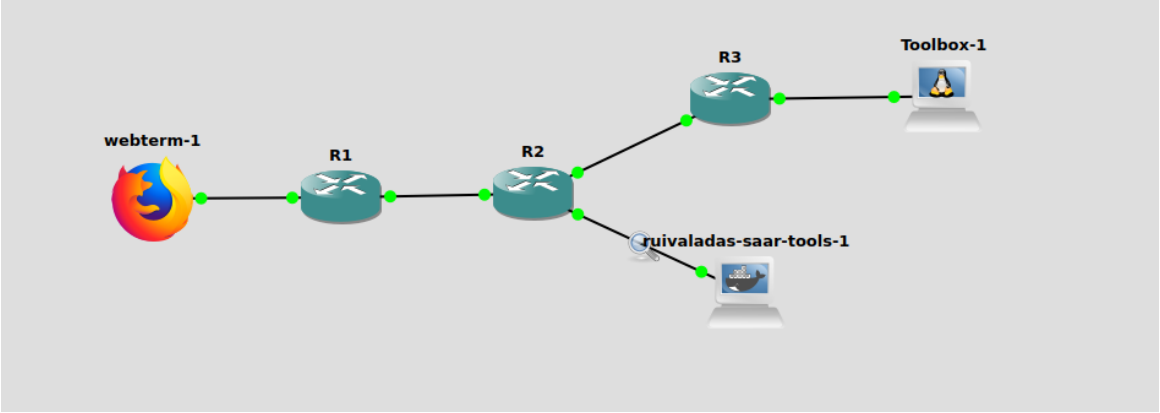


Figure 12: Network topology used for the RIP poisoning attack

2.6.2 Attack and Countermeasure

The attacker sent forged RIPv2 Response messages in order to poison the routing tables of the routers. These messages advertised a route to the legitimate Web server network through the attacker, causing traffic from the browser to be redirected to the fake Web server.

As shown in Figure 13, Wireshark captured multiple RIPv2 Response packets sent to the multicast address `224.0.0.9`, which is used by RIPv2 routers. The captured packet advertises the network `10.0.0.0` with metric 5 and subnet mask `255.255.255.128`, showing that the attacker injected routing information into the network.

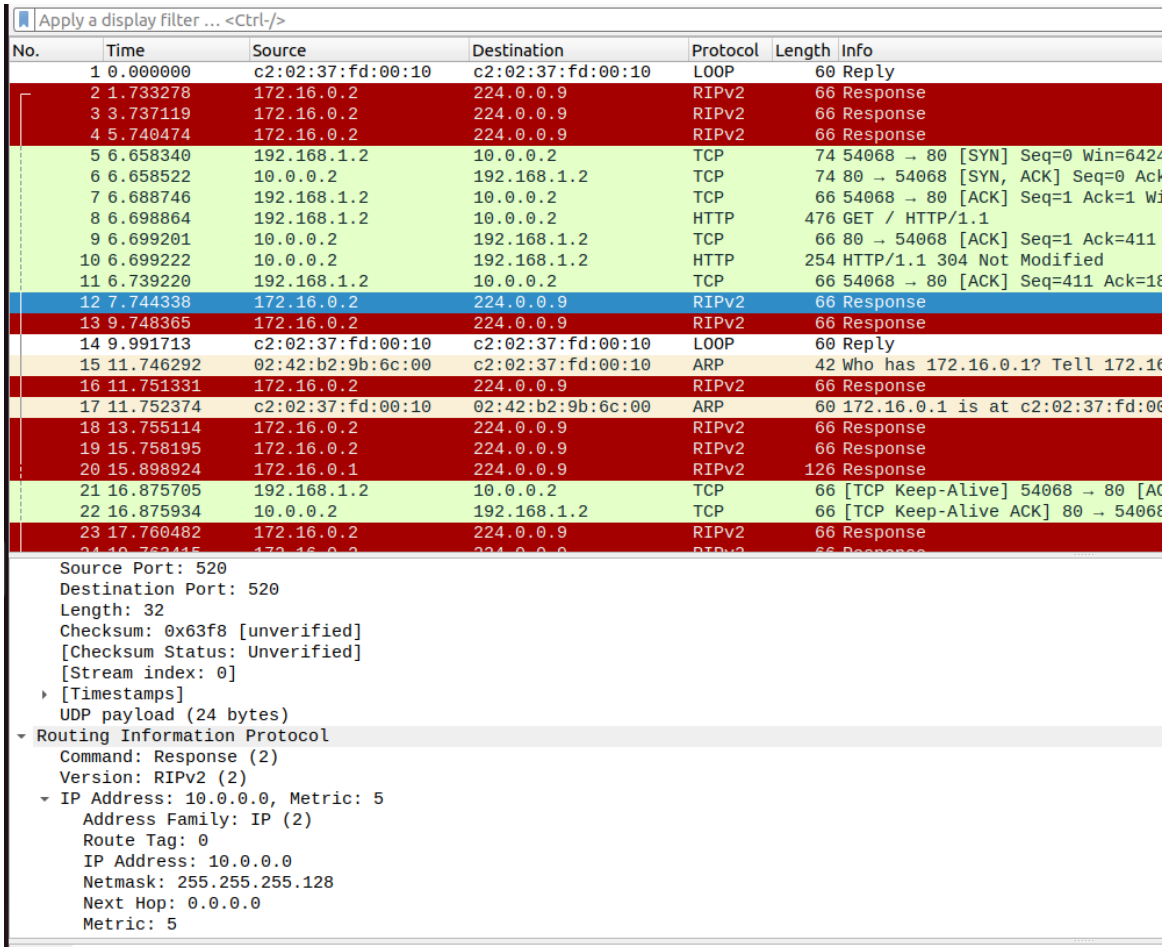


Figure 13: Forged RIPv2 response advertising a route to the target network

The effect of the attack was confirmed in the browser. As shown in Figure 14, the Web browser was redirected to the fake Web server, displaying the message *"FAKE SERVER - ATTACK SUCCESS"*.

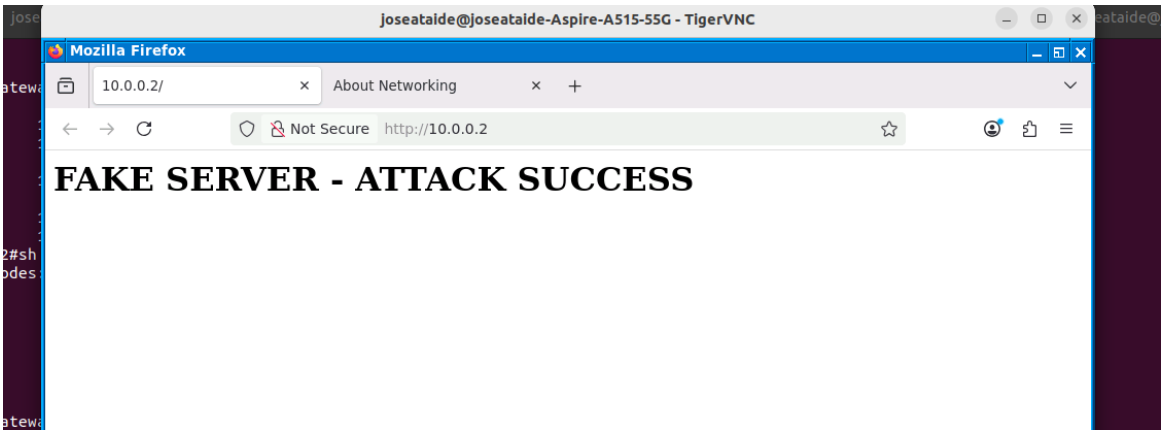


Figure 14: Browser redirected to the fake Web server

Additional Wireshark captures show TCP and HTTP communication between the browser and the fake server, confirming that the traffic was redirected after the routing tables were poisoned.

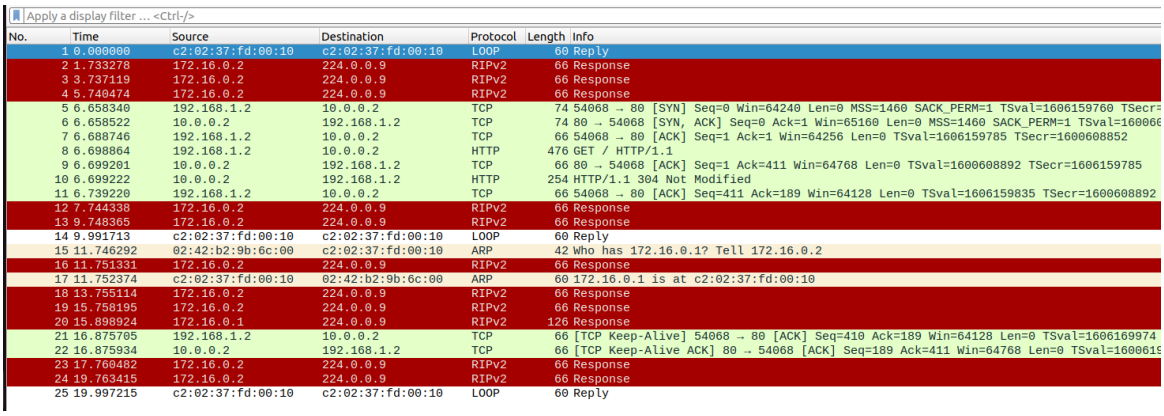


Figure 15: TCP and HTTP traffic redirected to the fake Web server

As a countermeasure, RIPv2 authentication can be enabled on the routers. With authentication, routers only accept RIP updates from trusted devices, preventing forged routing updates from being installed in the routing table.

2.6.3 Effectiveness of the Attack

The success of the attack depends on the relative location of the browser, legitimate Web server and fake Web server. Since the attacker is placed near the fake Web server, it can advertise a route that makes the fake server appear as a better path to the target network.

The prefix length advertised by the attacker is also important. A more specific prefix is preferred due to the longest prefix match rule. Therefore, if the attacker advertises a more specific route than the legitimate one, the routers may select the malicious route.

RIP auto-summary can reduce the effectiveness of the attack because it summarizes routes at classful network boundaries. When auto-summary is disabled, more specific routes can be advertised, making the attack more effective.

Figures 16 and 17 show repeated RIPv2 Response messages, confirming that the attacker continuously injected routing updates into the network.

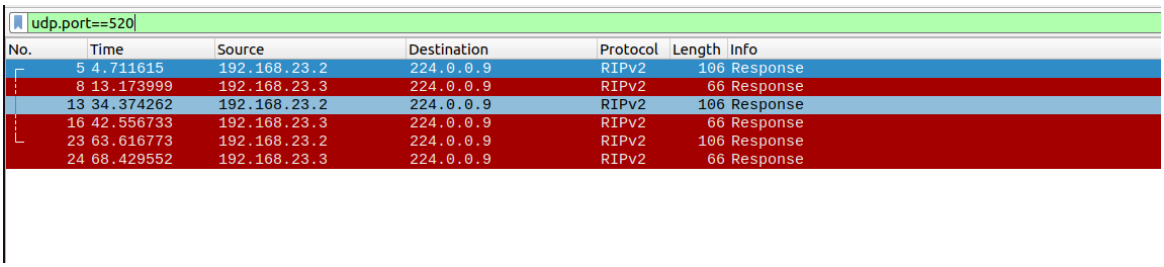


Figure 16: Filtered RIPv2 packets sent to multicast address 224.0.0.9

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
2	0.268414	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
3	3.282191	c2:02:37:fd:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: R2 Port ID: FastEthernet0/0
4	3.481026	c2:03:38:22:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: R3 Port ID: FastEthernet0/0
5	4.711615	192.168.23.2	224.0.0.9	RIPv2	106	Response
6	9.998962	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
7	10.275370	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
8	13.173999	192.168.23.3	224.0.0.9	RIPv2	66	Response
9	20.005237	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
10	20.270487	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
11	30.003637	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
12	30.270107	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
13	34.374262	192.168.23.2	224.0.0.9	RIPv2	106	Response
14	40.001557	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
15	40.272070	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
16	42.556733	192.168.23.3	224.0.0.9	RIPv2	66	Response
17	49.995916	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
18	50.267336	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
19	60.000765	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
20	60.272327	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
21	63.284522	c2:02:37:fd:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: R2 Port ID: FastEthernet0/0
22	63.475356	c2:03:38:22:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: R3 Port ID: FastEthernet0/0
23	63.616773	192.168.23.2	224.0.0.9	RIPv2	106	Response
24	68.429552	192.168.23.3	224.0.0.9	RIPv2	66	Response
25	69.998044	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
26	70.272005	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply
27	79.995333	c2:02:37:fd:00:00	c2:02:37:fd:00:00	LOOP	60	Reply
28	80.267995	c2:03:38:22:00:00	c2:03:38:22:00:00	LOOP	60	Reply

Figure 17: Repeated RIPv2 responses observed during the attack

2.6.4 AI Prediction vs Experimental Results

The AI predicted that the attacker would be more successful when advertising a more specific prefix, because routers prefer the longest prefix match. It also predicted that disabling RIP auto-summary would make the attack more effective, since the malicious route would not be summarized.

This prediction matched the experimental behavior. The captured RIP packets show that the attacker advertised specific routing information, and the browser traffic was redirected to the fake Web server. Therefore, the observed results confirm that prefix length and auto-summary directly influence route selection.

2.6.5 Feasibility

This attack is feasible in networks using RIP without authentication. Since RIP accepts routing updates from neighboring devices, an attacker can inject forged routes and redirect traffic.

However, in modern networks, RIP is less common and authentication can prevent this attack. Therefore, the attack is mainly feasible in poorly configured or legacy networks.

3 Firewalls

3.1 Classical firewalls versus Zone-Based Policy Firewalls

3.1.1 Firewall Validation

The firewall must enforce the following rules:

1. Block all traffic from INSIDE to OUTSIDE;
2. Allow only ICMP and HTTP traffic from OUTSIDE to the server;
3. Deny access to the firewall interfaces.

3.1.2 Classical Firewall (CBAC)

In the classical firewall, traffic filtering is based on ACLs combined with stateful inspection rules. As shown in the slides, inspection rules only track sessions and do not impose restrictions by themselves.

Initially, the configuration allowed access to multiple server ports because the ACL was too permissive. This confirms that **inspect rules do not have an implicit deny**, meaning that unwanted traffic may still pass if not explicitly blocked.

After correcting the ACLs, the observed behavior was:

1. Only HTTP and ICMP traffic to the server was allowed;
2. All other ports were classified as filtered by NMAP;
3. No traffic from INSIDE to OUTSIDE was possible due to explicit deny rules.

3.1.3 ZBPF

In ZBPF, interfaces are assigned to zones and policies are applied between zones. By default, traffic between different zones is dropped unless a policy is defined.

After configuration, the firewall behavior was:

1. Only explicitly permitted traffic (HTTP and ICMP to the server) was allowed;
2. All other inter-zone traffic was blocked by default;
3. Access to firewall interfaces was denied (controlled via the self zone).

Therefore, both firewalls satisfy the specification, but ZBPF enforces it more naturally due to its default deny model.

3.2 NMAP Operation

NMAP is used to discover hosts and scan ports by actively probing the network.

3.2.1 Host Discovery

When using `nmap -sn`, NMAP determines if hosts are active by sending:

1. ICMP Echo requests;
2. TCP SYN packets to common ports (e.g., port 80);
3. ARP requests in local networks.

A host is considered active if it replies to any of these probes.

3.2.2 Port Scanning

For port scanning, NMAP typically uses a TCP SYN scan:

1. SYN-ACK response ⇒ port is open;
2. RST response ⇒ port is closed;
3. No response or ICMP error ⇒ port is filtered.

This method is efficient because it does not complete the TCP connection, allowing fast scanning of multiple ports.

3.3 CBAC vs ZBPF

The main differences between CBAC and ZBPF are:

1. **Configuration model:** CBAC is interface-based and relies on ACLs plus inspection rules, while ZBPF is zone-based and uses zone-pairs, class-maps, and policy-maps.
2. **Default behavior:** In CBAC, traffic may be allowed unless explicitly denied by ACLs. In ZBPF, traffic between zones is denied by default unless a policy allows it.
3. **Stateful inspection:** Both support stateful inspection, but in CBAC it depends on inspect rules applied to interfaces, whereas in ZBPF it is integrated into policy actions (inspect, pass, drop).
4. **Security and control:** ZBPF provides stricter and more modular control, since all inter-zone traffic must be explicitly defined. CBAC is less structured and more prone to configuration errors.

In conclusion, although both approaches can enforce the required policy, ZBPF offers a more robust, scalable, and secure solution.

3.4 Protecting a Campus Network using ZBPF

3.4.1 Network Topology

The network is divided into four security zones: PR1, PR2, DMZ, and OUT, as shown in Figure 18.

The firewall is implemented on a Cisco 7200 router using Zone-Based Policy Firewall (ZBPF). The DMZ contains Web, Email, and DNS servers, while PR1 and PR2 represent private networks.

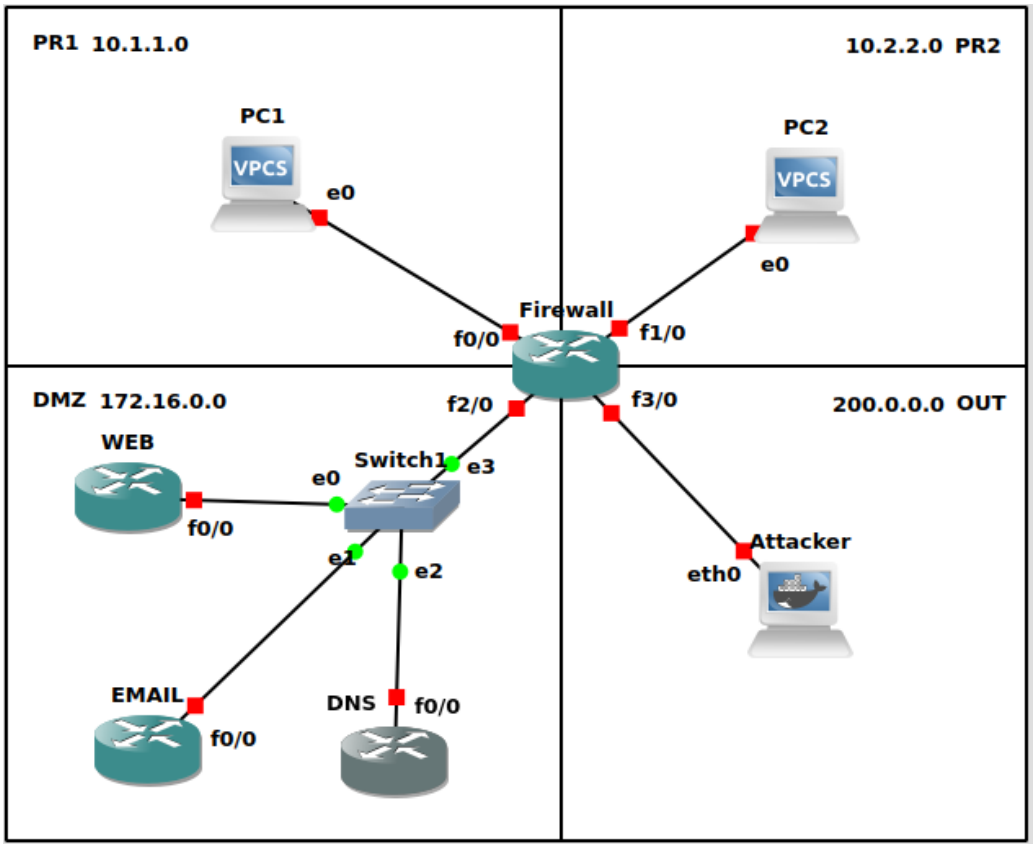


Figure 18: Campus network protected by a ZBPF firewall

3.4.2 DMZ Server Configuration

Three routers were configured to simulate the services hosted in the DMZ zone: a Web server, a DNS server, and an Email server.

Web Server The Web server was configured on the router with IP address `172.16.0.10`. HTTP and HTTPS services were enabled using the Cisco IOS embedded HTTP server functionality.

```
ip http server
ip http secure-server
username admin privilege 15 secret cisco
ip http authentication local
```

This configuration allowed the firewall policies for Web browsing (HTTP and HTTPS) to be validated using NMAP scans and TCP connection attempts.

DNS Server The DNS server was configured on the router with IP address `172.16.0.30`. The Cisco IOS DNS service was enabled and static host mappings were defined:

```
ip dns server
ip host webserver 172.16.0.10
ip host mailserver 172.16.0.20
```

This configuration provided basic DNS resolution functionality within the DMZ network and allowed DNS-related firewall rules to be tested.

Email Server Cisco IOS routers do not provide a complete Email server implementation supporting SMTP, POP3, and IMAP services. Therefore, the Email server was simulated using a router configured to provide reachable TCP services for firewall validation purposes.

```
banner motd # EMAIL SERVER #
ip http server
```

Although this configuration does not implement a real mail server, it is sufficient for validating the firewall policies related to Email traffic. The objective of the experiment was to verify that the firewall correctly permits or blocks the required protocols and ports, rather than deploying a production-ready Email infrastructure.

3.4.3 Firewall Configuration

The firewall was implemented using Cisco's Zone-Based Policy Firewall (ZBPF) model. In this approach, interfaces are assigned to security zones and traffic between zones is controlled through zone-pairs associated with policy-maps.

The topology was divided into four security zones:

- **PR1** – First private network;
- **PR2** – Second private network;
- **DMZ** – Demilitarized zone hosting public services;
- **OUT** – External network.

Each firewall interface was associated with the appropriate security zone. Since ZBPF denies inter-zone traffic by default, explicit policies had to be defined for all permitted communications.

The firewall policies were implemented using a modular approach based on:

1. ACLs identifying the destination servers;
2. Protocol-oriented class-maps identifying the allowed services;
3. Hierarchical class-maps combining destination and protocol matching;

4. Policy-maps defining the action applied to matching traffic.

For example, access to the DMZ Web server was conceptually separated into two components:

- an ACL identifying the Web server host;
- a protocol class-map grouping HTTP, HTTPS, and ICMP.

These components were then combined into a higher-level class-map and applied through a zone-pair policy.

This modular structure improves readability, scalability, and maintainability when compared with large monolithic ACLs containing both addressing and protocol logic simultaneously.

The private zones used addresses belonging to the range `10.0.0.0/8`. To allow communication with the OUT zone, NAT overload (PAT) was configured on the firewall. Internal addresses were translated to the public address of the OUT interface, allowing multiple private hosts to share a single external address.

The DMZ zone contained three simulated servers:

- a Web server (`172.16.0.10`);
- an Email server (`172.16.0.20`);
- a DNS server (`172.16.0.30`).

The Web server was configured using Cisco IOS embedded HTTP and HTTPS services. The DNS server used the Cisco IOS DNS service with static host mappings. Since Cisco IOS routers do not provide a complete Email server implementation, the Email server was simulated to validate firewall policies related to SMTP, POP3, and IMAP traffic.

The following zone-pairs were configured:

- PR1 → OUT;
- PR2 → OUT;
- PR1 → DMZ;
- PR2 → DMZ;
- OUT → DMZ;
- DMZ → OUT.

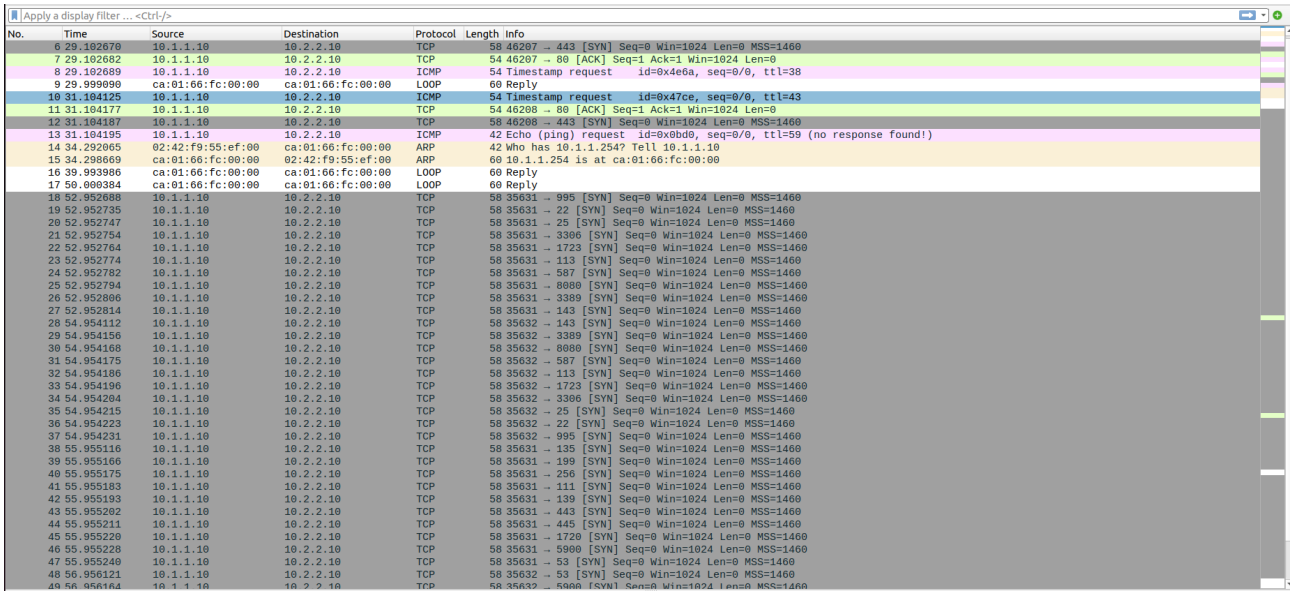
All traffic not explicitly permitted by the configured policies was denied by the firewall through the default ZBPF behavior.

The complete Cisco IOS firewall configuration is included in Appendix A.

3.4.4 Testing and Validation

The firewall policies were validated using NMAP and Wireshark.

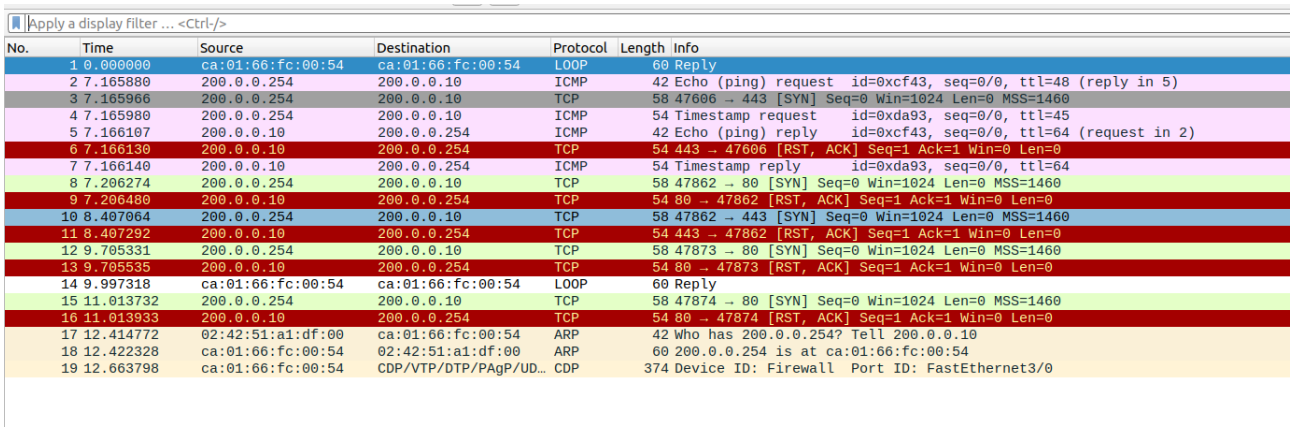
PR1 to PR2 Communication between private zones was blocked. NMAP scans returned no responses, confirming inter-zone isolation.



No.	Time	Source	Destination	Protocol	Length	Info
6	29.102670	10.1.1.10	10.2.2.10	TCP	58	46207 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
7	29.102682	10.1.1.10	10.2.2.10	TCP	54	46207 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
8	29.102689	10.1.1.10	10.2.2.10	ICMP	54	Timestamp request id=0x4e6a, seq=0/0, ttl=38
9	29.999090	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
10	31.104125	10.1.1.10	10.2.2.10	ICMP	54	Timestamp request id=0x47ce, seq=0/0, ttl=43
11	31.104177	10.1.1.10	10.2.2.10	TCP	54	46208 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0
12	31.104187	10.1.1.10	10.2.2.10	TCP	58	46208 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	31.104195	10.1.1.10	10.2.2.10	ICMP	42	Echo (ping) request id=0x0bd0, seq=0/0, ttl=59 (no response found!)
14	34.292065	02:42:f9:55:ef:00	ca:01:66:fc:00:00	ARP	42	Who has 10.1.1.254? Tell 10.1.1.10
15	34.298669	ca:01:66:fc:00:00	02:42:f9:55:ef:00	ARP	60	10.1.1.254 is at ca:01:66:fc:00:00
16	39.993986	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
17	50.000384	ca:01:66:fc:00:00	ca:01:66:fc:00:00	LOOP	60	Reply
18	52.952688	10.1.1.10	10.2.2.10	TCP	58	35631 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	52.952735	10.1.1.10	10.2.2.10	TCP	58	35631 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	52.952747	10.1.1.10	10.2.2.10	TCP	58	35631 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
21	52.952754	10.1.1.10	10.2.2.10	TCP	58	35631 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	52.952764	10.1.1.10	10.2.2.10	TCP	58	35631 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	52.952774	10.1.1.10	10.2.2.10	TCP	58	35631 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
24	52.952782	10.1.1.10	10.2.2.10	TCP	58	35631 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
25	52.952794	10.1.1.10	10.2.2.10	TCP	58	35631 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	52.952806	10.1.1.10	10.2.2.10	TCP	58	35631 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
27	52.952814	10.1.1.10	10.2.2.10	TCP	58	35631 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	54.954112	10.1.1.10	10.2.2.10	TCP	58	35632 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
29	54.954156	10.1.1.10	10.2.2.10	TCP	58	35632 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	54.954168	10.1.1.10	10.2.2.10	TCP	58	35632 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	54.954175	10.1.1.10	10.2.2.10	TCP	58	35632 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	54.954186	10.1.1.10	10.2.2.10	TCP	58	35632 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
33	54.954196	10.1.1.10	10.2.2.10	TCP	58	35632 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	54.954204	10.1.1.10	10.2.2.10	TCP	58	35632 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	54.954215	10.1.1.10	10.2.2.10	TCP	58	35632 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
36	54.954223	10.1.1.10	10.2.2.10	TCP	58	35632 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	54.954231	10.1.1.10	10.2.2.10	TCP	58	35632 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	55.955116	10.1.1.10	10.2.2.10	TCP	58	35631 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	55.955166	10.1.1.10	10.2.2.10	TCP	58	35631 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
40	55.955175	10.1.1.10	10.2.2.10	TCP	58	35631 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	55.955183	10.1.1.10	10.2.2.10	TCP	58	35631 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
42	55.955193	10.1.1.10	10.2.2.10	TCP	58	35631 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
43	55.955202	10.1.1.10	10.2.2.10	TCP	58	35631 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
44	55.955211	10.1.1.10	10.2.2.10	TCP	58	35631 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
45	55.955220	10.1.1.10	10.2.2.10	TCP	58	35631 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
46	55.955228	10.1.1.10	10.2.2.10	TCP	58	35631 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	55.955240	10.1.1.10	10.2.2.10	TCP	58	35631 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
48	56.956121	10.1.1.10	10.2.2.10	TCP	58	35632 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	56.956164	10.1.1.10	10.2.2.10	TCP	58	35632 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Figure 19: NMAP scan from PR1 to PR2 showing no replies, confirming inter-zone isolation enforced by the firewall

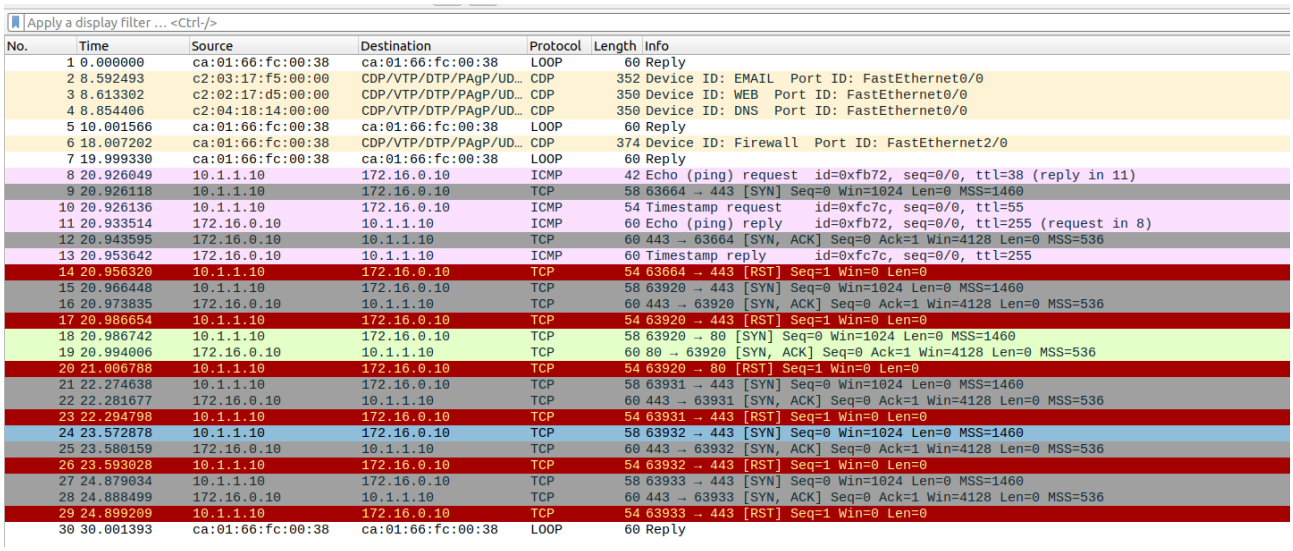
PR1 to OUT NAT operation was validated by scanning external hosts. Wireshark captures confirmed source address translation using the firewall public IP.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:66:fc:00:54	ca:01:66:fc:00:54	LOOP	60	Reply
2	7.165880	200.0.0.254	200.0.0.10	ICMP	42	Echo (ping) request id=0xcf43, seq=0/0, ttl=48 (reply in 5)
3	7.165966	200.0.0.254	200.0.0.10	TCP	58	47606 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
4	7.165980	200.0.0.254	200.0.0.10	ICMP	54	Timestamp request id=0xda93, seq=0/0, ttl=45
5	7.166107	200.0.0.10	200.0.0.254	ICMP	42	Echo (ping) reply id=0xcf43, seq=0/0, ttl=64 (request in 2)
6	7.166130	200.0.0.10	200.0.0.254	TCP	54	443 → 47606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	7.166140	200.0.0.10	200.0.0.254	ICMP	54	Timestamp reply id=0xda93, seq=0/0, ttl=64
8	7.206274	200.0.0.254	200.0.0.10	TCP	58	47862 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
9	7.206480	200.0.0.10	200.0.0.254	TCP	54	80 → 47862 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	8.407064	200.0.0.254	200.0.0.10	TCP	58	47862 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	8.407292	200.0.0.10	200.0.0.254	TCP	54	443 → 47862 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12	9.705331	200.0.0.254	200.0.0.10	TCP	58	47873 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	9.705535	200.0.0.10	200.0.0.254	TCP	54	80 → 47873 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14	9.997318	ca:01:66:fc:00:54	ca:01:66:fc:00:54	LOOP	60	Reply
15	11.013732	200.0.0.254	200.0.0.10	TCP	58	47874 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	11.013933	200.0.0.10	200.0.0.254	TCP	54	80 → 47874 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	12.414772	02:42:51:a1:df:00	ca:01:66:fc:00:54	ARP	42	Who has 200.0.0.254? Tell 200.0.0.10
18	12.422328	ca:01:66:fc:00:54	02:42:51:a1:df:00	ARP	60	200.0.0.254 is at ca:01:66:fc:00:54
19	12.663798	ca:01:66:fc:00:54	CDP/VTP/DTP/PAgP/UD…	CDP	374	Device ID: Firewall Port ID: FastEthernet3/0

Figure 20: Traffic from PR1 to the OUT zone showing NAT translation and access only to permitted services

PR1 to DMZ Only HTTP and HTTPS services were reachable on the DMZ Web server, while all other ports were filtered.



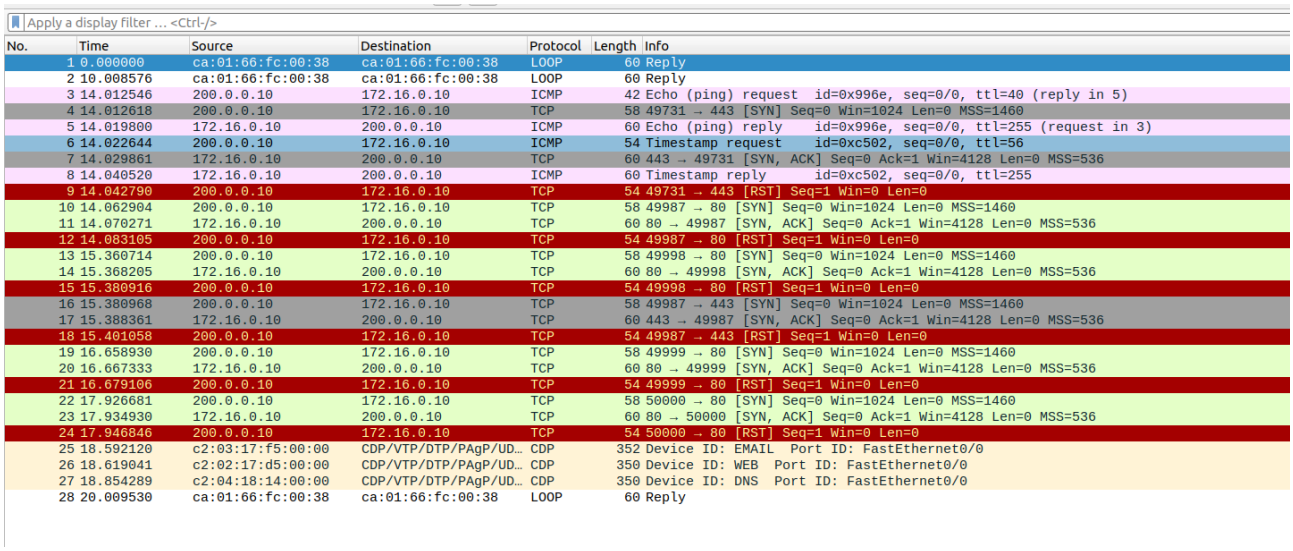
Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
2	8.592493	c2:03:17:f5:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	352	Device ID: EMAIL Port ID: FastEthernet0/0
3	8.613302	c2:02:17:d5:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	350	Device ID: WEB Port ID: FastEthernet0/0
4	8.854406	c2:04:18:14:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	350	Device ID: DNS Port ID: FastEthernet0/0
5	10.001566	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
6	18.007202	ca:01:66:fc:00:38	CDP/VTP/DTP/PAgP/UD…	CDP	374	Device ID: Firewall Port ID: FastEthernet2/0
7	19.999330	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
8	20.926049	10.1.1.10	172.16.0.10	ICMP	42	Echo (ping) request id=0xfb72, seq=0/0, ttl=38 (reply in 11)
9	20.926118	10.1.1.10	172.16.0.10	TCP	58	63664 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	20.926136	10.1.1.10	172.16.0.10	ICMP	54	Timestamp request id=0xfc7c, seq=0/0, ttl=55
11	20.933514	172.16.0.10	10.1.1.10	ICMP	60	Echo (ping) reply id=0xfb72, seq=0/0, ttl=255 (request in 8)
12	20.943595	172.16.0.10	10.1.1.10	TCP	60	443 → 63664 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
13	20.953642	172.16.0.10	10.1.1.10	ICMP	60	Timestamp reply id=0xfc7c, seq=0/0, ttl=255
14	20.956320	10.1.1.10	172.16.0.10	TCP	54	63664 → 443 [RST] Seq=1 Win=0 Len=0
15	20.966448	10.1.1.10	172.16.0.10	TCP	58	63920 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	20.973835	172.16.0.10	10.1.1.10	TCP	60	443 → 63920 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
17	20.986654	10.1.1.10	172.16.0.10	TCP	54	63920 → 443 [RST] Seq=1 Win=0 Len=0
18	20.986742	10.1.1.10	172.16.0.10	TCP	58	63920 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	20.994006	172.16.0.10	10.1.1.10	TCP	60	80 → 63920 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
20	21.006788	10.1.1.10	172.16.0.10	TCP	54	63920 → 80 [RST] Seq=1 Win=0 Len=0
21	22.274638	10.1.1.10	172.16.0.10	TCP	58	63931 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	22.281677	172.16.0.10	10.1.1.10	TCP	60	443 → 63931 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
23	22.294798	10.1.1.10	172.16.0.10	TCP	54	63931 → 443 [RST] Seq=1 Win=0 Len=0
24	23.572878	10.1.1.10	172.16.0.10	TCP	58	63932 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
25	23.580159	172.16.0.10	10.1.1.10	TCP	60	443 → 63932 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
26	23.593028	10.1.1.10	172.16.0.10	TCP	54	63932 → 443 [RST] Seq=1 Win=0 Len=0
27	24.879034	10.1.1.10	172.16.0.10	TCP	58	63933 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	24.888499	172.16.0.10	10.1.1.10	TCP	60	443 → 63933 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
29	24.899209	10.1.1.10	172.16.0.10	TCP	54	63933 → 443 [RST] Seq=1 Win=0 Len=0
30	30.001393	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply

Figure 21: NMAP scan from PR1 to the DMZ showing only HTTP and HTTPS services accessible

OUT to Private Zones Connections from the OUT zone to PR1 and PR2 were unsuccessful, confirming that private networks are protected from external access.

OUT to DMZ Only the permitted public services were accessible from the OUT zone.



Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
2	10.008576	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply
3	14.012546	200.0.0.10	172.16.0.10	ICMP	42	Echo (ping) request id=0x996e, seq=0/0, ttl=40 (reply in 5)
4	14.012618	200.0.0.10	172.16.0.10	TCP	58	49731 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	14.019800	172.16.0.10	200.0.0.10	ICMP	60	Echo (ping) reply id=0x996e, seq=0/0, ttl=255 (request in 3)
6	14.022644	200.0.0.10	172.16.0.10	ICMP	54	Timestamp request id=0xc502, seq=0/0, ttl=56
7	14.029861	172.16.0.10	200.0.0.10	TCP	60	443 → 49731 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
8	14.040520	172.16.0.10	200.0.0.10	ICMP	60	Timestamp reply id=0xc502, seq=0/0, ttl=255
9	14.042790	200.0.0.10	172.16.0.10	TCP	54	49731 → 443 [RST] Seq=1 Win=0 Len=0
10	14.062904	200.0.0.10	172.16.0.10	TCP	58	49987 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	14.070271	172.16.0.10	200.0.0.10	TCP	60	80 → 49987 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
12	14.083105	200.0.0.10	172.16.0.10	TCP	54	49987 → 80 [RST] Seq=1 Win=0 Len=0
13	15.360714	200.0.0.10	172.16.0.10	TCP	58	49998 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	15.368205	172.16.0.10	200.0.0.10	TCP	60	80 → 49998 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
15	15.380916	200.0.0.10	172.16.0.10	TCP	54	49998 → 80 [RST] Seq=1 Win=0 Len=0
16	15.380968	200.0.0.10	172.16.0.10	TCP	58	49987 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
17	15.388361	172.16.0.10	200.0.0.10	TCP	60	443 → 49987 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
18	15.401058	200.0.0.10	172.16.0.10	TCP	54	49987 → 443 [RST] Seq=1 Win=0 Len=0
19	16.658930	200.0.0.10	172.16.0.10	TCP	58	49999 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	16.667333	172.16.0.10	200.0.0.10	TCP	60	80 → 49999 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
21	16.679106	200.0.0.10	172.16.0.10	TCP	54	49999 → 80 [RST] Seq=1 Win=0 Len=0
22	17.926681	200.0.0.10	172.16.0.10	TCP	58	50000 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	17.934930	172.16.0.10	200.0.0.10	TCP	60	80 → 50000 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
24	17.946846	200.0.0.10	172.16.0.10	TCP	54	50000 → 80 [RST] Seq=1 Win=0 Len=0
25	18.592120	c2:03:17:f5:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	352	Device ID: EMAIL Port ID: FastEthernet0/0
26	18.619041	c2:02:17:d5:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	350	Device ID: WEB Port ID: FastEthernet0/0
27	18.854289	c2:04:18:14:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	350	Device ID: DNS Port ID: FastEthernet0/0
28	20.009530	ca:01:66:fc:00:38	ca:01:66:fc:00:38	LOOP	60	Reply

Figure 22: Access from the OUT zone to the DMZ restricted to publicly allowed services

3.4.5 AI-Based ZBPF Proposal and Comparison

An AI tool was initially used to propose a ZBPF configuration for the specified firewall policy. The proposed solution implemented most firewall rules correctly and produced the expected filtering behavior during testing. In particular, the AI-generated configuration correctly enforced:

- isolation between PR1 and PR2;
- restricted access from OUT to the private zones;
- controlled access to DMZ services;
- NAT overload for private networks;
- stateful inspection between security zones.

However, the AI-generated configuration relied primarily on large ACLs containing both addressing information and protocol definitions simultaneously. For example, individual ACL entries directly specified:

- source and destination addresses;
- transport protocols;
- destination service ports.

Although functionally correct, this approach results in configurations that are less modular and harder to maintain as the number of services and zones increases.

During the experiment, it became evident that a more structured ZBPF design can be achieved by separating:

1. host identification (ACLs);
2. protocol grouping (protocol class-maps);
3. policy logic (hierarchical class-maps and policy-maps).

This methodology, presented during the lectures, leverages the full capabilities of ZBPF and produces cleaner and more scalable firewall policies. Instead of defining large ACLs with complete filtering logic, ACLs are used only to identify servers or networks, while protocol class-maps group related services such as HTTP, HTTPS, DNS, SMTP, POP3, and ICMP.

The combined use of nested class-maps and policy-maps results in a configuration that is easier to understand, reuse, and extend.

Despite these architectural differences, both implementations enforce the same effective firewall behavior. Experimental validation using NMAP and Wireshark confirmed that the implemented firewall correctly satisfied all required security policies.

3.4.6 Conclusion

The experimental results confirm that the ZBPF firewall correctly enforces inter-zone isolation, controlled access to DMZ services, NAT operation, and secure firewall management.

3.5 Defense against DoS Attacks

This laboratory exercise explored the use of Cisco Zone-Based Policy Firewall (ZBPF) mechanisms to mitigate two common Denial of Service (DoS) attacks:

- ICMP Flood attack
- TCP SYN Flood attack

The experiments were performed using the `hping3` tool to generate malicious traffic towards the protected server while Wireshark captures and IOS inspection commands were used to analyze the behavior of the firewall.

3.5.1 ZBPF Configuration

The firewall configuration was based on a Zone-Based Policy Firewall architecture. Separate class maps were created for ICMP and HTTP traffic and combined into a single policy map.

```
ip access-list extended TO_IN_HOST
 permit ip any host 222.10.10.1
```

```
class-map type inspect match-all ICMP_CLASSMAP
 match access-group name TO_IN_HOST
 match protocol icmp
```

```
class-map type inspect match-all WWW_CLASSMAP
 match access-group name TO_IN_HOST
 match protocol http
```

```
policy-map type inspect DOS_POLICYMAP

 class type inspect WWW_CLASSMAP
  inspect
  police rate 128000 burst 8000

 class type inspect ICMP_CLASSMAP
  inspect
  police rate 128000 burst 8000

 class class-default
  drop log
```

The policy was then applied to the zone pair connecting the outside network to the protected internal network.

```
policy exists on zp OUT-TO-IN
 Zone-pair: OUT-TO-IN

  Service-policy inspect : DOS-POLICYMAP
```

Figure 23: DOS_POLICYMAP applied to the OUT_TO_IN zone pair

3.5.2 ICMP Flood Attack

The ICMP flood attack was generated using:

```
hping3 -1 --flood 222.10.10.1
```

This attack continuously sends ICMP Echo Request packets towards the victim with the objective of exhausting processing resources or saturating the network link.

Wireshark captures show a large number of ICMP Echo Request packets being transmitted towards the protected server.

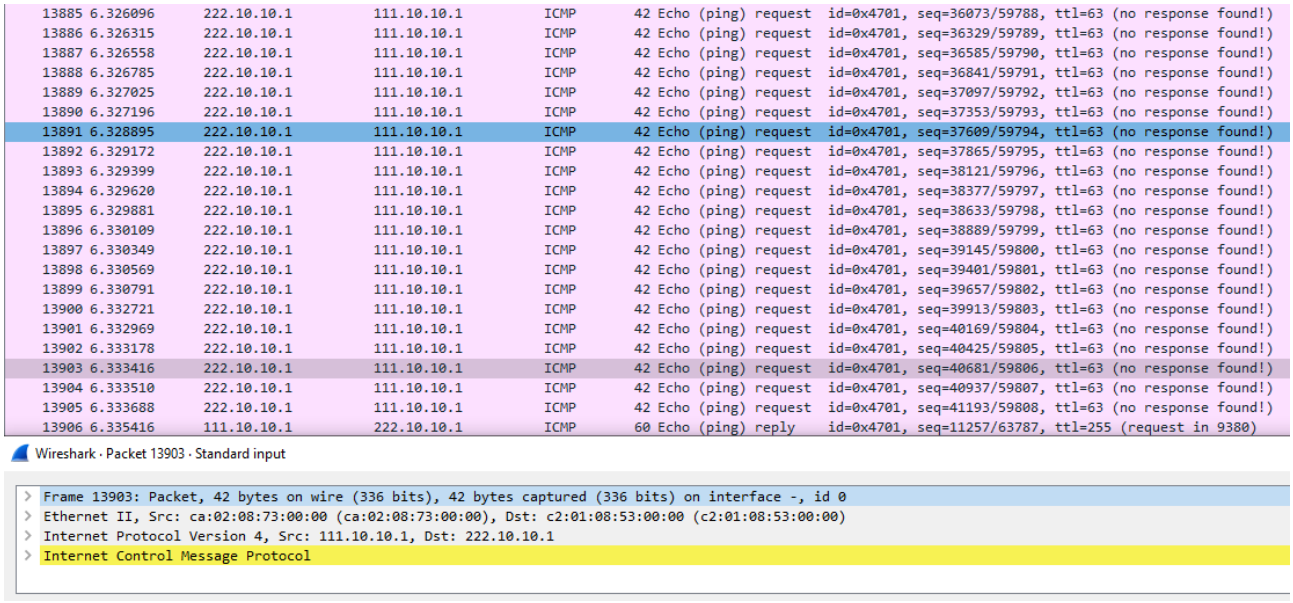


Figure 24: Wireshark capture of the ICMP flood attack

The ZBPF firewall inspected the ICMP traffic and enforced the configured policing policy. IOS inspection statistics confirmed that the firewall was actively processing ICMP packets.

```
Class-map: ICMP-CLASSMAP (match-all)
  Match: access-group name TO-IN-HOST
  Match: protocol icmp
```

Figure 25: ICMP traffic identified by the firewall class-map

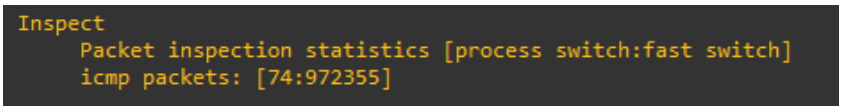


Figure 26: Firewall inspection statistics during the ICMP flood attack

The command:

```
show policy-map type inspect zone-pair OUT_TO_IN
```

confirmed that the firewall was inspecting and policing the attack traffic.

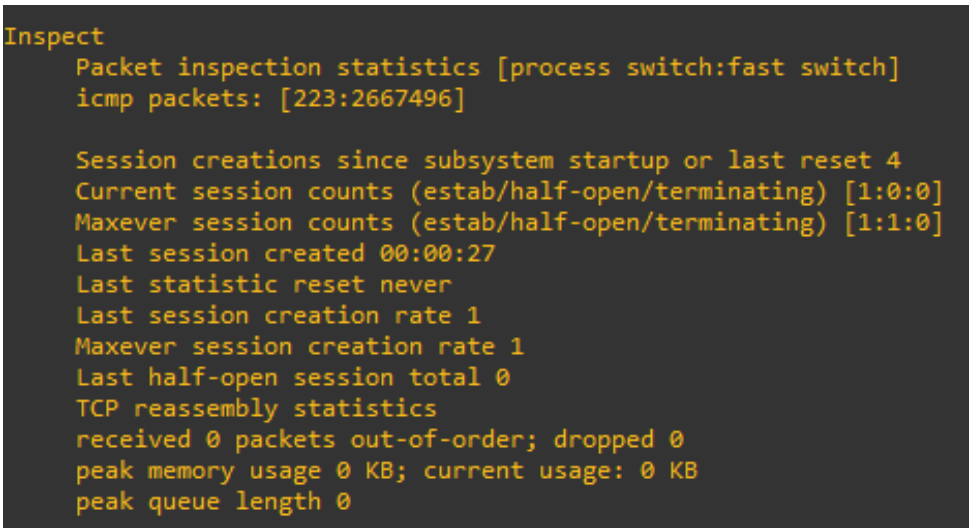


Figure 27: ZBPF inspection counters during the attack

The `police rate` parameter controls the maximum permitted bandwidth, while the `burst` parameter defines how much traffic may temporarily exceed the configured rate. Smaller values increase protection but may also drop legitimate traffic, whereas larger values improve throughput but reduce the effectiveness of the mitigation.

3.5.3 TCP SYN Flood Attack

The TCP SYN flood attack was performed using:

```
hping3 -S -p 80 --flood --rand-source 222.10.10.1
```

This attack generates a large number of TCP SYN packets with random spoofed source addresses. The victim server attempts to establish TCP sessions by replying with SYN-ACK packets, but the final ACK packet of the TCP three-way handshake never arrives.

As a consequence, the server accumulates many half-open TCP sessions, which may eventually exhaust available resources.

To mitigate this attack, a parameter map was configured:

```
parameter-map type inspect PARMAP
 tcp synwait-time 3
```

The parameter map reduces the TCP SYN wait timeout, allowing the firewall to remove half-open sessions more aggressively.

Wireshark captures show repeated TCP SYN packets directed at the server, followed by retransmissions and connection resets generated during the mitigation process.

483	13:04:03.231364	111.10.1…	222.10.1…	TCP	54	48706 → 80 [SYN] Seq=0 Win=512 Len=0
484	13:04:03.231387	111.10.1…	222.10.1…	TCP	54	48936 → 80 [SYN] Seq=0 Win=512 Len=0
485	13:04:03.231410	111.10.1…	222.10.1…	TCP	54	50191 → 80 [SYN] Seq=0 Win=512 Len=0
486	13:04:03.236676	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2789 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
487	13:04:03.241767	111.10.1…	222.10.1…	TCP	54	50300 → 80 [SYN] Seq=0 Win=512 Len=0
488	13:04:03.241817	111.10.1…	222.10.1…	TCP	54	50301 → 80 [SYN] Seq=0 Win=512 Len=0
489	13:04:03.241842	111.10.1…	222.10.1…	TCP	54	51156 → 80 [SYN] Seq=0 Win=512 Len=0
490	13:04:03.241865	111.10.1…	222.10.1…	TCP	54	51586 → 80 [SYN] Seq=0 Win=512 Len=0
491	13:04:03.241888	111.10.1…	222.10.1…	TCP	54	51615 → 80 [SYN] Seq=0 Win=512 Len=0
492	13:04:03.248214	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2790 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
493	13:04:03.262700	111.10.1…	222.10.1…	TCP	54	51806 → 80 [SYN] Seq=0 Win=512 Len=0
494	13:04:03.262794	111.10.1…	222.10.1…	TCP	54	51807 → 80 [SYN] Seq=0 Win=512 Len=0
495	13:04:03.268698	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2791 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
496	13:04:03.273147	111.10.1…	222.10.1…	TCP	54	53147 → 80 [SYN] Seq=0 Win=512 Len=0
497	13:04:03.273196	111.10.1…	222.10.1…	TCP	54	53208 → 80 [SYN] Seq=0 Win=512 Len=0
498	13:04:03.273221	111.10.1…	222.10.1…	TCP	54	53209 → 80 [SYN] Seq=0 Win=512 Len=0
499	13:04:03.279168	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2792 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
500	13:04:03.283600	111.10.1…	222.10.1…	TCP	54	55388 → 80 [SYN] Seq=0 Win=512 Len=0
501	13:04:03.290055	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2793 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
502	13:04:03.294241	111.10.1…	222.10.1…	TCP	54	55389 → 80 [SYN] Seq=0 Win=512 Len=0
503	13:04:03.294298	111.10.1…	222.10.1…	TCP	54	55390 → 80 [SYN] Seq=0 Win=512 Len=0
504	13:04:03.294328	111.10.1…	222.10.1…	TCP	54	56996 → 80 [SYN] Seq=0 Win=512 Len=0
505	13:04:03.294357	111.10.1…	222.10.1…	TCP	54	56997 → 80 [SYN] Seq=0 Win=512 Len=0
506	13:04:03.294406	111.10.1…	222.10.1…	TCP	54	58008 → 80 [SYN] Seq=0 Win=512 Len=0
507	13:04:03.300583	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2794 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
508	13:04:03.312608	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2795 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
509	13:04:03.324526	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2796 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
510	13:04:03.336505	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2797 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
511	13:04:03.348515	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2798 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
512	13:04:03.359988	c2:01:08…	c2:01:08…	LOOP	60	Reply
513	13:04:03.370463	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2799 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
514	13:04:03.384414	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2800 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
515	13:04:03.396496	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2801 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
516	13:04:03.408374	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2802 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
517	13:04:03.420374	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2803 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
518	13:04:03.436350	222.10.1…	111.10.1…	TCP	60	[TCP Retransmission] 80 → 2804 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=536
519	13:04:09.490571	111.10.1…	222.10.1…	TCP	60	58056 → 80 [SYN] Seq=0 Win=512 Len=0
520	13:04:09.494030	111.10.1…	222.10.1…	TCP	60	58057 → 80 [SYN] Seq=0 Win=512 Len=0

Figure 28: Wireshark capture of the TCP SYN flood attack

The captures demonstrate that the ZBPF firewall actively monitored the TCP state and helped protect the server by removing incomplete TCP sessions more rapidly.

3.5.4 Analysis of the Attacks and Mitigation

The ICMP flood attack attempts to overwhelm the victim using excessive ICMP traffic. The ZBPF firewall mitigated this attack by inspecting ICMP packets and enforcing rate limiting policies using the `police` command.

The TCP SYN flood attack is more sophisticated because it abuses the TCP connection establishment mechanism. By using spoofed source addresses, the attacker prevents the completion of the TCP handshake, forcing the server to maintain many half-open sessions.

The ZBPF firewall mitigated this attack using both traffic policing and TCP session inspection. The parameter map reduced the SYN timeout period, allowing the firewall to remove stale sessions more quickly.

Overall, the experiments demonstrate that Zone-Based Policy Firewalls provide effective protection against several classes of DoS attacks by combining:

- Stateful packet inspection
- Traffic classification
- Rate limiting
- TCP session management

Although these protections significantly improve resilience, very large-scale distributed attacks may still overwhelm network resources before filtering occurs. Therefore, firewall-based mitigation should be combined with additional protections such as upstream filtering, anti-spoofing policies, and distributed DDoS mitigation services.

4 AAA

4.1 AAA with TACACS+

4.1.1 AAA Configuration

AAA was enabled on the router using the IOS `aaa new-model` command. A local administrative user was initially configured to guarantee fallback access in case the TACACS+ server became unavailable.

```
username admin privilege 15 password 0 admin
enable password cisco
aaa new-model
```

The TACACS+ server was then configured on the router:

```
tacacs server Server-T
 address ipv4 10.0.0.1
 key mysecret123
```

Authentication, authorization, and accounting policies were configured as follows:

```
aaa authentication login default group tacacs+ local
aaa authorization exec default group tacacs+ local
aaa authorization commands 15 default group tacacs+ local
aaa accounting exec default start-stop group tacacs+
```

Remote access through Telnet was enabled on the VTY lines:

```
line vty 0 4
 login authentication default
 transport input telnet
```

On the TACACS+ server, users and permissions were configured in the file:

```
/etc/tacacs+/tac_plus.conf
```

The administrative user was configured with privilege level 15:

```
user = gns3 {
    login = des HASH

    service = exec {
        priv-lvl = 15
    }

    cmd = .* {
        permit .*
    }
}
```

A restricted user was also configured with limited command authorization:

```
user = readonly {
    login = cleartext "gns3"

    service = exec {
        priv-lvl = 1
    }

    cmd = show {
        permit .*
    }
}
```

4.1.2 Authentication, Authorization and Accounting

AAA separates administrative access control into three distinct functions.

Authentication Authentication was implemented using TACACS+ server-based login validation. When a Telnet session was initiated, R1 forwarded the supplied username and password to the TACACS+ server for verification.

Authentication was validated by successfully logging into the router using the centralized TACACS+ user accounts.

```
root@TelnetClient:~# telnet 222.10.10.254

Username: gns3
Password:

R1#
```

Authorization Authorization policies were configured on the TACACS+ server to provide different privilege levels to different users.

The user `gns3` received privilege level 15 and full administrative access, while the `readonly` user was restricted to monitoring commands only.

Authorization was validated by attempting privileged configuration commands using the restricted account.

```
R1>enable
Password:

R1#configure terminal
Command authorization failed
```

Accounting Accounting was configured to log TACACS+ administrative activity on the AAA server.

Session information and executed administrative actions were recorded in:

```
/var/log/tac_plus.acct
```

The generated logs include usernames, timestamps, session start and stop events, and command execution information.

```
# Created by Henry-Nicolas Tourneur
# See man(5) tac_plus.conf for more details

accounting file = /var/log/tac_plus.acct

key = mysecret123

user = DEFAULT {
    login = PAM
    service = ppp protocol = ip {}
}

user = gns3 {
    login = des 0dmbjUvhfS0jk

    service = exec {
        priv-lvl = 15
    }

    cmd = .* {
        permit .*
    }
}

user = readonly {
    login = cleartext "gns3"

    service = exec {
        priv-lvl = 1
    }

    cmd = show {
```

```
        permit .*
    }
}

group = admin {
    default service = permit

    service = exec {
        priv-lvl = 15
    }
}

group = read-only {
    service = exec {
        priv-lvl = 15
    }

    cmd = show {
        permit .*
    }

    cmd = write {
        permit term
    }

    cmd = dir {
        permit .*
    }

    cmd = admin {
        permit .*
    }

    cmd = terminal {
        permit .*
    }

    cmd = more {
        permit .*
    }

    cmd = exit {
        permit .*
    }

    cmd = logout {
        permit .*
    }
}
```

4.1.3 Password Configuration

Two different types of passwords were configured during the exercise.

The first was the TACACS+ shared secret, used to encrypt communication between the router and the TACACS+ server:

```
key = mysecret123
```

This same key was configured on both the router and the AAA server.

The second type corresponds to user authentication passwords. Initially, cleartext passwords were used, but later DES-encrypted hashes were configured.

```
tac_pwd
```

The generated hash was then inserted into the TACACS+ configuration:

```
login = des HASH
```

Authentication tests confirmed that users could still successfully log into the router using the original plaintext password, while the TACACS+ server stored only the DES hash.

4.1.4 Wireshark Analysis

Wireshark was used to capture and analyze TACACS+ traffic exchanged between the router and the AAA server.

TACACS+ communication uses TCP port 49. The packet captures show the establishment of TCP sessions followed by TACACS+ authentication, authorization, and accounting exchanges.

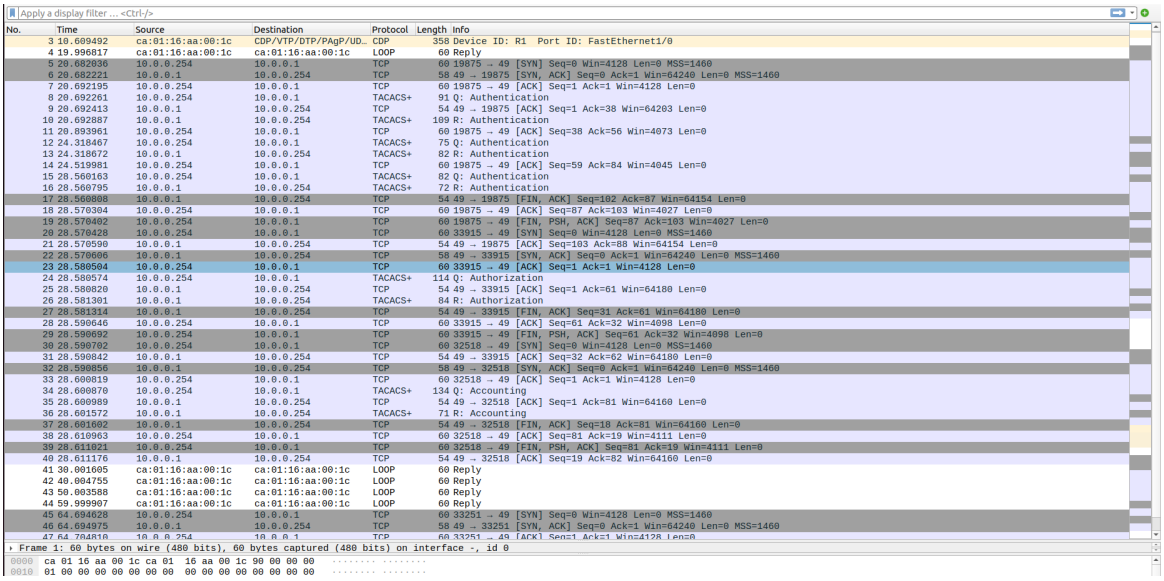


Figure 29: Wireshark capture of TACACS+ authentication traffic

Authentication packets were generated when users attempted to log into the router. Authorization packets were exchanged when IOS requested permission for command execution and privilege assignment. Accounting packets were generated when user sessions started or ended.

Because TACACS+ encrypts packet payloads, Wireshark decryption required the shared secret configured between the router and the TACACS+ server.

4.1.5 Benefits of Server-Based AAA

Compared to local AAA, server-based AAA provides centralized management of user accounts and permissions across multiple devices.

This approach simplifies administration, improves scalability, and enables detailed auditing of administrative activity. TACACS+ also provides granular command authorization, allowing different users to receive different privilege levels and command permissions.

Overall, centralized AAA provides significantly better security, control, and accountability than locally configured authentication alone.

4.2 AAA with Radius

4.2.1 Radius Configuration

AAA was enabled on the router using the IOS `aaa new-model` command. A local administrative user was also configured to guarantee fallback access if the Radius server became unavailable.

```
username admin privilege 15 password 0 admin
enable password cisco
aaa new-model
```

The Radius server was then configured on the router:

```
radius server Server-R
 address ipv4 10.0.0.1 auth-port 1812 acct-port 1813
 key myradiuskey
```

Authentication, authorization, and accounting policies were configured as follows:

```
aaa authentication login default group radius local
aaa authorization exec default group radius local
aaa accounting exec default start-stop group radius
```

Remote access through Telnet was enabled on the VTY lines:

```
line vty 0 4
 login authentication default
 transport input telnet
```

On the Radius server, users were configured in the file:

```
/etc/freeradius/3.0/users
```

The predefined users `bob` and `alice` were configured with password `gns3`.

The shared secret used between the Radius client and server was configured in:

```
/etc/freeradius/3.0/clients.conf
```

4.2.2 Authentication, Authorization and Accounting

Authentication Authentication was implemented using Radius server-based login validation. When a Telnet session was initiated, R1 forwarded the supplied credentials to the Radius server for verification.

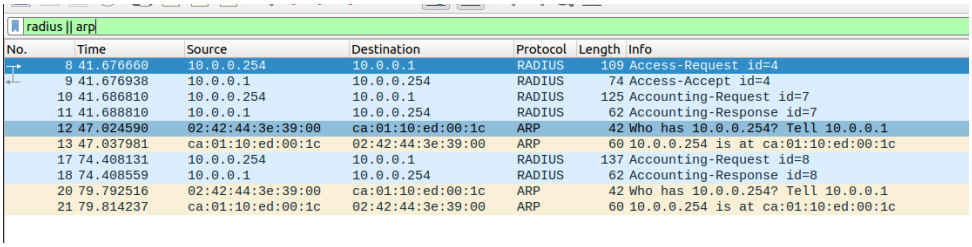
Authentication was validated by successfully logging into the router using the centralized Radius user accounts.

```
root@TelnetClient:~# telnet 222.10.10.254
Trying 222.10.10.254...
Connected to 222.10.10.254.
Escape character is '^]'.


User Access Verification

Username: bob
Password:
Hello, bob

R1>exit
Connection closed by foreign host.
root@TelnetClient:~# telnet 222.10.10.254
Trying 222.10.10.254...
Connected to 222.10.10.254.
Escape character is '^'.

User Access Verification

Username: alice
Password:
Hello, alice

R1>exit
Connection closed by foreign host.
root@TelnetClient:~#
```

Authorization Wireshark captures show that Radius exchanges mainly consist of authentication and accounting messages between R1 and the Radius server.

During user login, the router sends an `Access-Request` message containing information such as the username, NAS IP address, and authentication parameters. The Radius server then replies with either an `Access-Accept` or `Access-Reject` message, depending on the validity of the supplied credentials.

At the beginning and end of a successful administrative session, accounting messages are exchanged. The router sends `Accounting-Request` packets to report session start and stop events, while the server replies with `Accounting-Response` messages.



Unlike TACACS+, Radius generates relatively few messages during the session itself. Most Radius traffic is observed only during login and logout operations. TACACS+, on the other hand, generates additional authorization exchanges throughout the session because IOS commands can be individually authorized by the TACACS+ server.

Accounting Accounting was configured to log administrative activity on the Radius server. Session information was stored in:

`/var/log/freeradius/radacct/`

The generated logs contain information such as usernames, timestamps, session start and stop events, and NAS information.

```
Sat May  9 15:24:52 2026
    Acct-Session-Id = "00000001"
    User-Name = "bob"
    Acct-Authentic = RADIUS
    Acct-Status-Type = Start
    NAS-Port = 2
    NAS-Port-Id = "tty2"
    NAS-Port-Type = Virtual
    Service-Type = NAS-Prompt-User
    NAS-IP-Address = 10.0.0.254
    Acct-Delay-Time = 0
    Event-Timestamp = "May  9 2026 15:24:52 UTC"
    Tmp-String-9 = "ai:"
    Acct-Unique-Session-Id = "5f17e8201ba1e8d959a54331f562dbd0"
    Timestamp = 1778340292

Sat May  9 15:25:00 2026
    Acct-Session-Id = "00000001"
    User-Name = "bob"
    Acct-Authentic = RADIUS
    Acct-Terminate-Cause = User-Request
    Acct-Session-Time = 8
    Acct-Status-Type = Stop
    NAS-Port = 2
    NAS-Port-Id = "tty2"
    NAS-Port-Type = Virtual
```

```
    Service-Type = NAS-Prompt-User
    NAS-IP-Address = 10.0.0.254
    Acct-Delay-Time = 0
    Event-Timestamp = "May  9 2026 15:25:00 UTC"
    Tmp-String-9 = "ai:"
    Acct-Unique-Session-Id = "5f17e8201ba1e8d959a54331f562dbd0"
    Timestamp = 1778340300

Sat May  9 15:25:07 2026
    Acct-Session-Id = "00000002"
    User-Name = "alice"
    Acct-Authentic = RADIUS
    Acct-Status-Type = Start
    NAS-Port = 2
    NAS-Port-Id = "tty2"
    NAS-Port-Type = Virtual
    Service-Type = NAS-Prompt-User
    NAS-IP-Address = 10.0.0.254
    Acct-Delay-Time = 0
    Event-Timestamp = "May  9 2026 15:25:07 UTC"
    Tmp-String-9 = "ai:"
    Acct-Unique-Session-Id = "683333e47c0de024a04e881f984026f2"
    Timestamp = 1778340307
```

4.2.3 Comparison Between TACACS+ and Radius

Although both TACACS+ and Radius provide centralized AAA services, important differences exist between the two protocols.

TACACS+ uses TCP port 49, while Radius uses UDP ports 1812 and 1813. TACACS+ encrypts the entire packet payload, whereas Radius encrypts only the password field.

Another important difference is that TACACS+ separates authentication, authorization, and accounting into independent processes and supports per-command authorization for IOS devices. Radius combines authentication and authorization and does not support granular IOS command authorization.

Radius is widely used for network access authentication, while TACACS+ is more commonly used for administrative access control in Cisco environments.

Overall, TACACS+ provides more granular administrative control, while Radius offers broader interoperability and simpler deployment.

4.3 802.1X Authentication

IEEE 802.1X authentication was implemented using a Cisco IOSvL2 switch as the authenticator, a Linux host running `wpa_supplicant` as the supplicant, and a FreeRADIUS server as the authentication server.

The switch ports were configured for 802.1X port-based authentication using RADIUS.

```
aaa new-model
aaa authentication dot1x default group radius

radius server AAAServer
 address ipv4 10.1.1.50 auth-port 1812 acct-port 1813
 key gns3

dot1x system-auth-control
```

The supplicant configuration used PEAP with MSCHAPv2 authentication:

```
network={
    key_mgmt=IEEE8021X
    eap=PEAP
    identity="bob"
    password="gns3"
    phase2="auth=MSCHAPV2"
}
```

Authentication was initiated using:

```
wpa_supplicant -Dwired -ieth0 -c/usr/wpa.conf
```

Successful authentication was confirmed on the Cisco switch through the authenticated session table.

```
Switch#sh authentication sessions

Interface    MAC Address    Method  Domain  Status Fg Session ID
Gi0/0        0242.9b27.a300 dot1x   DATA    Auth      0A0101640000000C0007766D

Session count = 1
```

Figure 30: Authenticated 802.1X session on the Cisco switch

4.3.1 Wireshark Analysis of EAP-PEAP Authentication

Wireshark was used to capture and analyze the EAP-PEAP authentication exchange between the supplicant and the switch.

The captures show the complete IEEE 802.1X authentication process, including:

- EAPOL-Start
- EAP Request Identity
- EAP Response Identity
- Protected EAP (PEAP)
- TLS handshake messages
- EAP Success

1	15:18:12.336932	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAPOL	18	Start
2	15:18:12.356056	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Request, Identity
3	15:18:12.356948	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	26	Response, Identity
4	15:18:12.404767	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Request, Protected EAP (EAP-PEAP)
5	15:18:12.407037	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	218	Client Hello
6	15:18:12.442912	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	1022	Request, Protected EAP (EAP-PEAP)
7	15:18:12.443276	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	24	Response, Protected EAP (EAP-PEAP)
8	15:18:12.474104	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	200	Server Hello, Certificate, Server Key Exchange, Server Hello Done
9	15:18:12.477726	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	154	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
10	15:18:12.519615	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	75	Change Cipher Spec, Encrypted Handshake Message
11	15:18:12.520202	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	EAP	24	Response, Protected EAP (EAP-PEAP)
12	15:18:12.565761	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	60	Application Data
13	15:18:12.566196	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	57	Application Data
14	15:18:12.610079	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	92	Application Data
15	15:18:12.610534	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	111	Application Data
16	15:18:12.664969	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	100	Application Data
17	15:18:12.666643	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	55	Application Data
18	15:18:12.711178	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	TLSv1.2	64	Application Data
19	15:18:12.712360	02:42:9b:27:a3:00	Nearest-non-TPMR-Bridge	TLSv1.2	64	Application Data
20	15:18:12.856284	0c:8b:65:46:00:00	Nearest-non-TPMR-Bridge	EAP	60	Success

Figure 31: Wireshark capture of the EAP-PEAP authentication process

The authentication process starts when the supplicant sends an EAPOL-Start frame after connecting to the switch port.

The switch, acting as the authenticator, responds with an EAP Request Identity message. The supplicant then replies with an EAP Response Identity containing the username used for authentication.

```
> Frame 3: Packet, 26 bytes on wire (208 bits), 26 bytes captured (208 bits) on interface -, id 0
> Ethernet II, Src: 02:42:9b:27:a3:00 (02:42:9b:27:a3:00), Dst: Nearest-non-TPMR-Bridge (01:80:c2:00:00:03)
> 802.1X Authentication
v Extensible Authentication Protocol
    Code: Response (2)
    Id: 1
    Length: 8
    Type: Identity (1)
    Identity: bob
```

Figure 32: EAP Response Identity packet containing the username used during authentication

The EAP Response Identity packet contains the username provided by the supplicant during the authentication exchange.

In this setup, the supplicant identified itself using the username `bob`, which was later validated by the FreeRADIUS server through PEAP/MSCHAPv2 authentication.

The switch forwards the authentication information to the FreeRADIUS server using the RADIUS protocol.

PEAP negotiation is then initiated in order to establish a TLS tunnel between the supplicant and the authentication server. The Wireshark captures show TLS handshake messages, including Client Hello and Change Cipher Spec.

After the TLS tunnel is established, MSCHAPv2 authentication takes place inside the encrypted channel.

Finally, the authentication process completes successfully with the EAP Success message, allowing the switch to authorize the port for normal network access.

4.3.2 Security Considerations

IEEE 802.1X authentication can mitigate several network attacks studied in Lab 1.1.

By requiring authentication before granting access to switch ports, unauthorized devices cannot easily connect to the network.

This mechanism helps mitigate:

- Unauthorized network access
- Rogue devices connected to switch ports
- Basic MAC spoofing attempts
- Passive credential sniffing attacks

The use of PEAP additionally protects authentication credentials by establishing an encrypted TLS tunnel between the supplicant and the authentication server.

However, IEEE 802.1X does not directly mitigate Denial of Service attacks such as ICMP flood or TCP SYN flood attacks studied previously.

4.3.3 Comparison Between AI Explanation and Observed Traffic

An AI tool was used to explain the EAP-PEAP authentication process implemented in the lab setup.

The AI explanation correctly identified the three main entities involved in IEEE 802.1X authentication:

- Supplicant
- Authenticator
- Authentication Server

The supplicant corresponds to the Linux host running `wpa_supplicant`, which initiates the authentication process and provides user credentials.

The authenticator corresponds to the Cisco switch, which controls access to the network port and forwards EAP messages between the supplicant and the RADIUS server.

The authentication server corresponds to the FreeRADIUS server, which validates credentials and decides whether authentication succeeds or fails.

The Wireshark captures confirmed the behavior described by the AI explanation. Observed traffic included EAPOL-Start, Identity exchange messages, PEAP negotiation, TLS handshake packets, and the final EAP Success message.

One important observation is that the switch does not authenticate the user directly. Instead, the switch acts only as an intermediary between the supplicant and the FreeRADIUS server. The actual authentication decision is performed exclusively by the RADIUS server.

Overall, the AI explanation was consistent with the observed protocol behavior and packet captures.

5 Conclusion

The laboratory activities performed throughout this project provided practical experience with several important network security mechanisms, attacks, and defensive technologies commonly used in modern infrastructures.

The first part of the project demonstrated that many traditional network attacks remain feasible in environments lacking proper security controls. MAC table overflow, DHCP spoofing, ARP poisoning, DNS spoofing, and RIP poisoning attacks were successfully reproduced and analyzed using Wireshark captures and Cisco IOS commands. The experiments also showed that modern countermeasures such as port security, DHCP snooping, Dynamic ARP Inspection, HTTPS/TLS protections, and routing authentication significantly reduce the effectiveness of these attacks.

The firewall experiments demonstrated the differences between Classical Firewalls (CBAC) and Zone-Based Policy Firewalls (ZBPF). Although both approaches can enforce security policies, ZBPF provides a more modular, scalable, and secure architecture through the use of zones, class-maps, policy-maps, and stateful inspection. The firewall configurations successfully enforced network segmentation, controlled access to DMZ services, and NAT operation.

The Denial of Service experiments showed how ICMP flood and TCP SYN flood attacks can affect network services and server resources. Wireshark captures confirmed the behavior of the attacks, including excessive ICMP traffic and the accumulation of half-open TCP sessions. The implemented ZBPF protections, based on traffic policing and TCP session management, effectively mitigated the attacks and reduced their impact on the protected server.

The AAA exercises highlighted the advantages of centralized authentication and authorization using TACACS+ and RADIUS. The experiments demonstrated how centralized AAA improves administrative control, scalability, and accountability when compared with local authentication mechanisms. IEEE 802.1X authentication further extended these concepts by enforcing port-based access control and protecting network access using EAP-PEAP authentication through a secure TLS tunnel.

Overall, the project demonstrated the importance of combining multiple layers of security protections in order to defend modern networks against both external and internal threats. Packet analysis using Wireshark and practical experimentation in GNS3 were essential to understand the behavior of protocols, attacks, and mitigation mechanisms at a detailed level.

The performed experiments reinforce the idea that network security depends not only on the existence of protection mechanisms, but also on their correct configuration, integration, and continuous monitoring.

A Firewall Configuration

This appendix contains the relevant Cisco IOS configuration commands used to implement the Zone-Based Policy Firewall (ZBPF) for the campus network topology.

A.1 Zone Definitions

```
conf t

zone security PR1
zone security PR2
zone security DMZ
zone security OUT
```

A.2 Interface Configuration and Zone Assignment

```
interface FastEthernet0/0
 ip address 10.1.1.254 255.255.255.0
 ip nat inside
 ip virtual-reassembly in
 zone-member security PR1

interface FastEthernet1/0
 ip address 10.2.2.254 255.255.255.0
 ip nat inside
 ip virtual-reassembly in
 zone-member security PR2

interface FastEthernet2/0
 ip address 172.16.0.254 255.255.255.0
 zone-member security DMZ

interface FastEthernet3/0
 ip address 200.0.0.254 255.255.255.0
 ip nat outside
 ip virtual-reassembly in
 zone-member security OUT
```

A.3 NAT Configuration

```
ip access-list standard NAT_INSIDE
 permit 10.0.0.0 0.255.255.255

ip nat inside source list NAT_INSIDE interface FastEthernet3/0 overload
```

A.4 Access Control Lists

A.4.1 PR to OUT

```
ip access-list extended PR_TO_OUT
 permit tcp any any eq www
 permit tcp any any eq 443
 permit udp any any eq domain
 permit icmp any any
```

A.4.2 PR to DMZ

```
ip access-list extended PR_TO_DMZ
 permit icmp any host 172.16.0.10
 permit icmp any host 172.16.0.20
 permit icmp any host 172.16.0.30

 permit tcp any host 172.16.0.10 eq www
 permit tcp any host 172.16.0.10 eq 443

 permit tcp any host 172.16.0.20 eq smtp
 permit tcp any host 172.16.0.20 eq pop3
 permit tcp any host 172.16.0.20 eq 143

 permit udp any host 172.16.0.30 eq domain
 permit tcp any host 172.16.0.30 eq domain
```

A.4.3 OUT to DMZ

```
ip access-list extended OUT_TO_DMZ
 permit icmp any host 172.16.0.10
 permit icmp any host 172.16.0.20
 permit icmp any host 172.16.0.30

 permit tcp any host 172.16.0.10 eq www
 permit tcp any host 172.16.0.10 eq 443

 permit tcp any host 172.16.0.20 eq smtp

 permit udp any host 172.16.0.30 eq domain
 permit tcp any host 172.16.0.30 eq domain
```

A.4.4 DMZ to OUT

```
ip access-list extended DMZ_TO_OUT
 permit icmp any any
 permit tcp any any eq smtp
 permit udp any any eq domain
 permit tcp any any eq domain
```

A.5 Class-Maps

```
class-map type inspect match-any PR_OUT_CLASS
 match access-group name PR_TO_OUT

class-map type inspect match-any PR_DMZ_CLASS
 match access-group name PR_TO_DMZ

class-map type inspect match-any OUT_DMZ_CLASS
 match access-group name OUT_TO_DMZ

class-map type inspect match-any DMZ_OUT_CLASS
 match access-group name DMZ_TO_OUT
```

A.6 Policy-Maps

```
policy-map type inspect PR_OUT_POLICY
 class type inspect PR_OUT_CLASS
  inspect
 class class-default
  drop

policy-map type inspect PR_DMZ_POLICY
 class type inspect PR_DMZ_CLASS
  inspect
 class class-default
  drop

policy-map type inspect OUT_DMZ_POLICY
 class type inspect OUT_DMZ_CLASS
  inspect
 class class-default
  drop

policy-map type inspect DMZ_OUT_POLICY
 class type inspect DMZ_OUT_CLASS
  inspect
 class class-default
  drop
```

A.7 Zone-Pairs

```
zone-pair security PR1_TO_OUT source PR1 destination OUT
 service-policy type inspect PR_OUT_POLICY

zone-pair security PR2_TO_OUT source PR2 destination OUT
 service-policy type inspect PR_OUT_POLICY

zone-pair security PR1_TO_DMZ source PR1 destination DMZ
 service-policy type inspect PR_DMZ_POLICY

zone-pair security PR2_TO_DMZ source PR2 destination DMZ
 service-policy type inspect PR_DMZ_POLICY

zone-pair security OUT_TO_DMZ source OUT destination DMZ
 service-policy type inspect OUT_DMZ_POLICY

zone-pair security DMZ_TO_OUT source DMZ destination OUT
 service-policy type inspect DMZ_OUT_POLICY
```

A.8 SSH Management Access

```
username admin privilege 15 secret cisco

ip domain-name lab.local

crypto key generate rsa
1024

line vty 0 4
 login local
 transport input ssh
```

A.9 Web Server Configuration

```
ip http server
ip http secure-server

username admin privilege 15 secret cisco
ip http authentication local
```

A.10 DNS Server Configuration

```
ip dns server

ip host webserver 172.16.0.10
ip host mailserver 172.16.0.20
```

A.11 Email Server Simulation

```
banner motd # EMAIL SERVER #

ip http server
```